NGUYEN DEVELOPMENT · PHPDBADMIN

Evolutionary Development — phpdbadmin

Timeboxes und Architekturentscheide ADR-001 bis ADR-011

Phase	04 Evolutionary Development
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	29.10.2026, 02:34 · baut auf Commit f1af885 + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. 04 — Evolutionary Development
2. Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
3. Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
4. Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
5. Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
6. Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
7. Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
8. Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
9. Anhang H — ADR-008 — Anmeldung ohne Passwort
10. Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
11. Anhang J — ADR-010 — Lizenz MIT
12. Anhang K — ADR-011 — PHP 8.5 als Mindestversion
13. Anhang L — ADR-012 — SSH-Tunnel über das System-ssh
14. Anhang M — Timebox 00 — Fundament
15. Anhang N — Timebox 01 — Katalog und Navigator
16. Anhang O — Timebox 02 — Datenraster und Filter
17. Anhang P — Timebox 03 — SQL-Editor und Befehlspalette → Alpha
18. Anhang Q — Timebox 04 — Bearbeiten mit Vorschau und Rückgängig
19. Anhang R — Timebox 06 — Export und Import im Hintergrund, Statistik
20. Anhang S — TB 07 — Konten, Rechte je Verbindung, Teilen, Protokoll
21. Anhang T — Timebox 09 — Server-Verwaltung
22. Anhang U — Timebox 11 — Schemavergleich, Laravel-Migration, Herd-Erkennung, SSH-Tunnel

Inhalt

04 — Evolutionary Development
Timeboxes
Architekturentscheide

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen
Kontext
Entscheid
Optionen
Konsequenzen

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber
Kontext
Entscheid
Optionen
Konsequenzen

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber
Kontext
Entscheid
Optionen
Konsequenzen

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf
Kontext
Entscheid
Optionen
Konsequenzen

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6
Kontext
Entscheid
Optionen
Konsequenzen

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel
Kontext
Entscheid
Optionen
Konsequenzen

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund
Kontext
Entscheid
Optionen
Konsequenzen

Anhang H — ADR-008 — Anmeldung ohne Passwort
Kontext
Entscheid
Optionen
Konsequenzen
Nachtrag TB 00 (28.10.2026)

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe
Kontext
Entscheid
Optionen
Konsequenzen

Anhang J — ADR-010 — Lizenz MIT
Kontext
Entscheid
Optionen
Konsequenzen

Anhang K — ADR-011 — PHP 8.5 als Mindestversion
Kontext
Entscheid
Optionen
Konsequenzen

Anhang L — ADR-012 — SSH-Tunnel über das System-ssh

Kontext
Optionen
Entscheid
Konsequenzen

Anhang M — Timebox 00 — Fundament

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Browser-Durchlauf
Entscheide in der Timebox
Commits
Offen
Gelernt

Anhang N — Timebox 01 — Katalog und Navigator

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

Anhang O — Timebox 02 — Datenraster und Filter

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

Anhang P — Timebox 03 — SQL-Editor und Befehlspalette → Alpha

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

Anhang Q — Timebox 04 — Bearbeiten mit Vorschau und Rückgängig

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Commits
Gelernt

Anhang R — Timebox 06 — Export und Import im Hintergrund, Statistik

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Messwerte
Abweichungen von den Wireframes
Tests
Commits
Offen
Gelernt

Anhang S — TB 07 — Konten, Rechte je Verbindung, Teilen, Protokoll

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Abweichungen von den Wireframes
Tests
Entscheide in der Timebox
Commits
Offen
Gelernt

Anhang T — Timebox 09 — Server-Verwaltung

Ziel
Umfang
Was entstanden ist
Abnahmekriterien

Gestaltungsentscheide (statt Wireframe-Abweichungen)
Entscheide in der Timebox
Tests
Commits
Offen
Gelernt

Anhang U — Timebox 11 — Schemavergleich, Laravel-Migration, Herd-Erkennung, SSH-Tunnel

Ziel
Umfang
Was entstanden ist
Abnahmekriterien
Gestaltungsentscheide
Entscheide in der Timebox
Tests
Commits
Offen
Gelernt

04 — Evolutionary Development

> **PDF der Phase:** evolutionary-development.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 04`).

Timebox-Protokolle (`timebox-nn-titel.md`) und Architekturentscheide (`adr/ADR-nnn-titel.md`: Kontext → Optionen → Entscheid → Konsequenzen).

Timeboxes

TB	Inhalt	Anforderungen	Stand
00 Fundament	Starter-Kit bereinigt (kein Passwort, kein Dashboard, kein Profil); Anmeldung mit Code per Mail und Passkey; Einladung; Ersteinrichtung per Konsole mit Link und Passkey-Onboarding; App-Datenbank; Verbindungen mit Test, Verschlüsselung und erlaubten Hosts; Dialekt-Schicht mit Vertragstest auf vier Datenbanken; lokales Prüfskript, Workflow sparsam	A-01 bis A-06, A-09, V-01 bis V-03, V-05, B-01 bis B-03, B-06, B-08	✅ 28.10.2026 — offen: Touch ID von Hand (A-03), erster Lauf des Workflows auf GitHub
01 Katalog und Navigator	Arbeitsbildschirm mit Navigator für vier Datenbanken, «alle Datenbanken», Strukturansicht mit Art je Spalte, Schreibschutz und Umgebung, Kontowarnung, Deutsch und Englisch	O-01, O-04, S-01, V-04, N-06, V-10, O-06, N-04	✅ 29.10.2026 — V-06 mit TB 07; N-04-Matrix noch nie gelaufen
02 Datenraster und Filter	Raster mit Blättern über den Schlüssel, NULL zuletzt, Filtern, Suche, Zählen, Fremdschlüssel in beide Richtungen, Zeilen-Detail, Reiter, Kürzel; erste Seite bei 1 Mio. Zeilen unter 20 ms	D-01, D-02, D-03, D-06, N-01, O-03, O-05, O-09	✅ 29.10.2026 — «?»-Taste von Hand offen
03 SQL-Editor und Befehlspalette	Zerleger je Dialekt, Ausführen mit Zeit- und Zeilengrenze, Abbrechen auf dem Server, SQLite im Arbeitsprozess, Editor mit Vervollständigung, Verlauf, gespeicherte Abfragen, gefährliche Anweisungen nur mit Bestätigung, Befehlspalette ⌘K	Q-01–Q-04, Q-07, Q-09, O-02	✅ 29.10.2026 → **Alpha** — Woche im Alltag offen (Hoang)
04 Bearbeiten mit Vorschau und Rückgängig	Änderungen sammeln, Vorschau aus `pretend()` mit Kaskaden, eine Transaktion auf allen vier, Rückgängig mit Konfliktprüfung, Feld-Editor je Art, Kopieren als TSV/Markdown/INSERT, Schreibschutz auf Zeit	D-04, D-05, D-07, D-09, E-05, N-03, V-11	✅ 29.10.2026
06 Export und Import	Export als CSV, JSON oder SQL in Paketen über den Schlüssel; Import von CSV mit Zuordnung und Probelauf, SQL-Datei mit eigenem Zerleger; Statistik nach Importen und auf Knopfdruck; Aufträge im Hintergrund, ohne Worker in der Anfrage mit Grenze	E-01, E-02, E-06, B-07	✅ 29.10.2026 — offen: Export einer Abfrage und gefilterter Zeilen
07 Konten, Rechte, Protokoll	Policy je Verbindung (`view`, `write`, `manage`, `share`); Teilen; Verwaltung der Konten und Einladungen; Protokoll mit Filtern, Maskierung und Frist; dazu Favoriten und «zuletzt geöffnet» (V-06)	B-04, A-07, V-09, A-05, A-06, V-06	✅ 29.10.2026 auf eigenem Zweig — offen: Browser-Durchlauf nach dem Zusammenführen

TB	Inhalt	Anforderungen	Stand
09 Server-Verwaltung	Übersicht, Prozesse beenden, Variablen, Konten und Rechte mit Vorschau, Wartung — vier Treiber, Dialekt-Vertrag erweitert, jede Änderung genau ein Eintrag im Protokoll	W-01 bis W-05	✅ 29.10.2026 auf eigenem Zweig, vorgezogen — offen: Browser-Durchlauf
11 Vergleich, Migration, Herd, SSH	Schemavergleich mit Änderungsskript im Dialekt des Ziels, auch zwischen zwei Treibern; Laravel-Migration aus einer Tabelle, auf einem anderen Treiber geprüft; Vorschläge aus Laravel Herd ohne Passwörter im Browser; Verbindungen über SSH-Tunnel mit dem System-ssh (ADR-012)	S-08, S-09, V-07, V-08	✅ 29.10.2026 auf eigenem Zweig — offen: Browser-Durchlauf, V-07 auf Hoangs Mac

Architekturentscheide

Ausformuliert in der Foundation am 28.10.2026 (Kontext → Optionen → Entscheid → Konsequenzen), Grundlage Feasibility und Solution-Architektur:

ADR	Entscheid	Status
ADR-001	Dialekt-Schicht: ein Vertrag, vier Umsetzungen	angenommen
ADR-002	Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber	angenommen
ADR-003	Vorschau aus `toSql()`, Anwenden je Treiber	angenommen
ADR-004	Schreibschutz in Schichten, eine Anweisung je Aufruf	angenommen
ADR-005	SQL-Editor CodeMirror 6	angenommen
ADR-006	Raster TanStack Virtual, Blättern über den Schlüssel	angenommen
ADR-007	Export und Import in Paketen im Hintergrund	angenommen
ADR-008	Anmeldung ohne Passwort	angenommen
ADR-009	App-Datenbank SQLite als Vorgabe	angenommen
ADR-010	Lizenz MIT	angenommen
ADR-011	PHP 8.5 als Mindestversion	angenommen (Hoang 28.10.2026)
ADR-012	SSH-Tunnel über das System-ssh	angenommen (TB 11, 29.10.2026)

Übernahme aus der Feasibility (erledigt in TB 00): Die Proben F-1, F-7 und Teile von F-2 sind Vertragstests der Dialekt-Schicht (`tests/Feature/Dialects`); die Probe-Seiten `routes/probe.php`, `resources/js/pages/probe/*` bleiben lokal hinter der Anmeldung, bis Raster und SQL-Editor sie ersetzen — die Raster-Probe ist seit TB 02 entfernt, die Editor-Probe folgt mit TB 03.

Anhang A — ADR-001 — Dialekt-Schicht: ein Vertrag, vier Umsetzungen

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-1, F-3)
Betrifft	Katalog, Struktur, SQL-Editor, Server-Verwaltung

Kontext

phpdbadmin soll vier Datenbanken mit derselben Bedienung abdecken (P-01). Laravel 13 liefert Verbindungen, Query Builder, Schema-Grammatiken und Introspektion für alle vier; die Feasibility zeigt, wo es nicht reicht: Check-Constraints, Trigger, Routinen, eigene Typen (`getTypes()` nur PostgreSQL), Prozesse, Rechte, Zeitlimits, Kontorechte. Ausserdem liefert MariaDB ohne Schemaangabe die Tabellen aller Datenbanken des Servers (3'603 statt 1'000).

Optionen

Option	Vorteile	Nachteile
A — eigene Abstraktion für alles	volle Kontrolle	baut nach, was Laravel schon kann und testet; viermal so viel Code
B — Laravel zuerst, Dialekt nur für Unterschiede	wenig eigener Code, Laravel-Updates kommen mit	Grenze zwischen Laravel und Dialekt muss klar sein
C — Paket eines Dritten (z. B. Adminer-Treiber)	viel fertig	fremde Struktur, Lizenzfrage, Bedienkonzept passt nicht

Entscheid

Option B. Ein Vertrag `App\Dialects\Dialect` mit vier Umsetzungen (`MySqlDialect`, `MariaDbDialect` als Ableitung, `PostgresDialect`, `SqliteDialect`). Was Laravel kann, ruft der Dienst direkt; der Dialekt liefert nur, was sich unterscheidet. Der Katalog übergibt **immer** das Schema.

Konsequenzen

- Jede Dialekt-Methode hat einen Vertragstest, der gegen alle vier Herd-Dienste läuft; die Proben aus `tests/Feature/Feasibility` werden dazu umgebaut.
- Neue Datenbanken (SQL Server, R-14) bekommen eine fünfte Umsetzung, die Oberfläche bleibt.
- Typnamen erscheinen im Original und mit einem gemeinsamen Oberbegriff.

Siehe Solution-Architektur · Feasibility

Anhang B — ADR-002 — Zielverbindungen je Anfrage, Grenzen und Abbruch je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-2, F-8)
Betrifft	ConnectionFactory, QueryRunner

Kontext

Zugangsdaten liegen verschlüsselt in der App-Datenbank. Jede Anfrage braucht eine Verbindung mit Zeitzone UTC, Zeitlimit und gegebenenfalls Schreibschutz. Die Feasibility zeigt grosse Unterschiede: MySQL begrenzt nur SELECT und bricht still ab (Ergebnis `1`), MariaDB und PostgreSQL melden 70100 bzw. 57014, SQLite kennt kein Zeitlimit; libpq hält ganze Ergebnisse ausserhalb des PHP-Speichers (+105 MB bei einer Million Zeilen).

Optionen

Option	Vorteile	Nachteile
A — Verbindungen in `config/database.php` eintragen	einfach	Neustart je Verbindung, Geheimnisse in Dateien
B — `DB::build()` je Anfrage aus der App-Datenbank	keine Dateien, Name je Verbindung und Konto, `DB::purge` danach	Verbindungsaufbau je Anfrage (lokal ~1 ms)
C — dauerhafte Verbindungen (Pool)	schneller	PHP-FPM hat keinen gemeinsamen Pool; Zustand zwischen Konten

Entscheid

Option B. `ConnectionFactory` baut je Anfrage mit `timezone`, Zeitlimit (`max_execution_time`, `max_statement_time`, `statement_timeout`) und Optionen. Zeilengrenze je Treiber: MySQL/MariaDB ungepuffert + `KILL QUERY`, PostgreSQL Server-Cursor (`DECLARE … CURSOR`, `FETCH`), SQLite streamt. SQLite-Abfragen aus dem Editor laufen im Arbeitsprozess mit Zeitgrenze. Abbrechen ist eine zweite Anfrage; ein Merker im Cache entscheidet über «abgebrochen», nicht der Fehlercode.

Konsequenzen

- Verbindungsfehler werden je Treiber zugeordnet (MySQL nach Code, PostgreSQL nach Meldungstext).
- Der Arbeitsprozess für SQLite braucht die Warteschlange oder einen Kindprozess; ohne beides gilt die PHP-Laufzeitgrenze.
- Erlaubte Hosts werden vor dem Verbinden geprüft (ADR-004, V-05).

Siehe Solution-Architektur · Feasibility

Anhang C — ADR-003 — Vorschau aus Blueprint::toSql(), Anwenden je Treiber

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-5)
Betrifft	SchemaEditor, ChangeSet

Kontext

Nichts soll ungesehen passieren (P-02). Die Feasibility belegt: `toSql()` ist auf allen vier genau das, was läuft; `pretend()` liefert für DML eine gute Vorschau, für SQLite-DDL aber eine kaputte. Naiv in einer Transaktion angewendet, wirft Laravel auf MySQL/MariaDB «There is no active transaction» (DDL beendet Transaktionen implizit), und **SQLite löscht beim Umbau einer Elterntabelle alle abhängigen Zeilen** (0 von 2), weil `PRAGMA foreign_keys` in einer Transaktion nicht wirkt.

Optionen

Option	Vorteile	Nachteile
A — alles in einer Transaktion	einfach, scheinbar sicher	auf drei von vier Treibern falsch, auf SQLite Datenverlust
B — Verfahren je Treiber	auf allen vier belegt korrekt (2 von 2 Kindern bleiben)	drei Wege, die Vorschau muss sie erklären
C — Strukturänderungen nur über eigenes SQL	keine Abhängigkeit von Laravel	baut die Grammatiken nach, die SQLite-Umbau schon können

Entscheid

Option B. Vorschau von Strukturänderungen aus `Blueprint::toSql()`, von Datenänderungen aus `pretend()`. Anwenden: MySQL/MariaDB Anweisung für Anweisung ohne Transaktion (Vorschau: «nicht rückrollbar»); PostgreSQL in einer Transaktion; SQLite nach sqlite.org: Fremdschlüssel ausserhalb aus, Umbau in der Transaktion, `foreign_key_check`, Commit, ein. Datenänderungen auf allen vier in einer Transaktion, vorher Sicherung für Rückgängig.

Konsequenzen

- Tests für Strukturänderungen immer mit Elterntabelle und abhängigen Zeilen.
- Bei MySQL/MariaDB meldet die Oberfläche nach einem Fehler, welche Schritte schon gelaufen sind.
- Lektion «SQLite-Tabellenumbau nie in eigener Transaktion» im Vault.

Siehe Solution-Architektur · Feasibility

Anhang D — ADR-004 — Schreibschutz in Schichten und eine Anweisung je Aufruf

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-7)
Betrifft	ReadOnlyGuard, QueryRunner, SQLite-Verbindungen

Kontext

Produktion ist standardmässig schreibgeschützt (V-04), und der SQL-Editor nimmt freies SQL. Ein Schreibschutz über eine Sitzungseinstellung allein lässt sich im Editor abschalten (`SET … READ WRITE`, `PRAGMA query_only = OFF`). Die Feasibility hat einen Wächter mit 5–12 Ausbruchsversuchen je Treiber geprüft: Bestand unverändert.

Optionen

Option	Vorteile	Nachteile
A — nur Lesekonto verlangen	das einzige echte Schloss	viele haben nur ein Konto; lokal unpraktisch
B — Schichten: Lesekonto empfohlen, Nur-lesen-Transaktion je Anweisung, vorbereitet, SQLite nur lesend geöffnet + Authorizer	belegt auf allen vier	jede Anweisung einzeln; Zerleger nötig
C — SQL vor dem Ausführen klassifizieren und Schreibendes ablehnen	ohne Transaktion	Parser-Rennen gegen jede Syntax; Funktionen mit Nebenwirkung

Entscheid

Option B. MySQL/MariaDB: `SET SESSION TRANSACTION READ ONLY` + Transaktion, immer zurückrollen; PostgreSQL: Transaktion + `SET TRANSACTION READ ONLY`, zurückrollen; SQLite: `Pdo\Sqlite::OPEN_READONLY` und `setAuthorizer()` (`ATTACH` nur im erlaubten Ordner). Anweisungen werden auf dem Server zerlegt und einzeln vorbereitet; `unprepared()` ist für Eingaben verboten. Konten mit Superuser- oder Dateirechten werden sichtbar gewarnt (V-10).

Konsequenzen

- Der Zerleger je Dialekt ist Voraussetzung (TB 03, Probe zu Beginn).
- Die Ausbruchsversuche aus `SecurityProbeTest` werden Abnahmetest von V-04.
- PHP 8.5 wird Mindestversion (ADR-011).

Siehe Solution-Architektur · Feasibility

Anhang E — ADR-005 — SQL-Editor: CodeMirror 6

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-4, Freigabe der Pakete durch Hoang)
Betrifft	Oberfläche SQL-Reiter

Kontext

Der Editor braucht Hervorhebung und Vervollständigung je Dialekt aus dem Schema (Q-01). Probeweise installiert: `codemirror` 6.0.2, `@codemirror/lang-sql` 6.10.0 (MIT, 17 Pakete). Belegt: Tabellen aus 1'002 Tabellen in höchstens 2 ms, Aliasse aufgelöst, dialekteigene Schlüsselwörter (`ILIKE` nur PostgreSQL); Chunk 425 KB (138 KB gzip).

Optionen

Option	Vorteile	Nachteile
A — CodeMirror 6	modular, MIT, Dialekte für alle vier, klein genug	eigene Vue-Hülle
B — Monaco	VS-Code-Gefühl	mehrere MB, SQL-Dialekte schwächer, Web-Worker
C — Eigenbau mit Textarea	keine Abhängigkeit	keine Vervollständigung, viel Arbeit

Entscheid

Option A, als eigener Chunk nur auf dem SQL-Reiter geladen.

Konsequenzen

- Sichtprüfung im Browser durch Hoang steht aus (Probe-Seite `/probe/editor`).
- Bei Updates der Pakete Lizenzliste neu erzeugen (ADR-010).

Siehe Solution-Architektur · Feasibility

Anhang F — ADR-006 — Raster: TanStack Virtual und Blättern über den Schlüssel

Feld	Wert
Status	Angenommen — bestätigt von Hoang am 28.10.2026 (Endlos scrollen statt Seiten zu 50)
Datum	28.10.2026 (Foundation; F-3)
Betrifft	Datenraster

Kontext

Tabellen mit Millionen Zeilen müssen sofort erscheinen (N-01). Gemessen: erste Seite über den Schlüssel 0,1–0,6 ms in der Datenbank, `OFFSET 900000` 18–232 ms; im Browser 281 ms für die erste Seite, davon fast alles Laravel und Herd; im DOM bleiben rund 25 Zeilen.

Optionen

Option	Vorteile	Nachteile
A — eigenes Raster mit `@tanstack/vue-virtual` (MIT, 34 KB Chunk)	volle Kontrolle über Bearbeiten, Zustände, Gestaltung	Raster selbst bauen
B — fertiges Datenraster (AG Grid Community u. a.)	viel fertig	schwer, eigene Bedienlogik, Gestaltung schwer anzugleichen
C — ohne Virtualisierung, Seiten zu 100	einfach	kein flüssiges Scrollen, Seitenwechsel

Entscheid

Option A. Seiten zu 200 Zeilen über den Schlüssel, die nächste vorgeladen; `OFFSET` nur ohne Primärschlüssel; erste Spalte fixiert, waagrecht scrollen nur hier.

Konsequenzen

- Geänderte, neue und zu löschende Zeilen sind Zustand im Browser bis «Anwenden» (D-04).
- Kartenliste unter 768 px (G-07).

Siehe Solution-Architektur · Feasibility

Anhang G — ADR-007 — Export und Import in Paketen im Hintergrund

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; F-8, F-3)
Betrifft	Transfer

Kontext

Gemessen bei einer Million Zeilen: `lazyById(5000)` 3–3,3 MB auf allen vier; `cursor()` gepuffert 62 MB (MySQL/MariaDB), PostgreSQL +105 MB im Prozess. Import von 100'000 Zeilen in Paketen zu 1'000: 143–502 ms. Nach dem Füllen fehlt MySQL/MariaDB die Statistik — Sortieren über den Index 300 statt 2 ms.

Optionen

Option	Vorteile	Nachteile
A — Pakete über den Schlüssel, im Hintergrund, Datei mit Frist	gleich auf allen vier, wenig Speicher	Warteschlange nötig (ohne Worker synchron mit Grenze)
B — Server-Werkzeuge (`mysqldump`, `pg_dump`)	schnell, vollständig	nicht überall installiert, Versionen müssen passen — nur für Sicherung (E-03)

Entscheid

Option A für Export und Import; Probelauf beim Import in einer zurückgerollten Transaktion; nach eigenen Importen `ANALYZE` (E-06). Server-Werkzeuge nur für Sicherung und Wiederherstellung (E-03, v1.1).

Konsequenzen

- Exportdateien unter `storage/app/private/transfers`, Frist 24 Stunden.
- Tabellen ohne Primärschlüssel exportieren über `cursor()` ungepuffert bzw. Server-Cursor.

Siehe Solution-Architektur · Feasibility

Anhang H — ADR-008 — Anmeldung ohne Passwort

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Vorgabe Hoang; F-6)
Betrifft	Anmeldung, Konten

Kontext

Vorgabe: kein Passwort, nur Passkey und Code per Mail; kein Dashboard, kein Profil. Fortify 1.41 registriert `POST /login` und die Passwortbestätigung unabhängig von den Features; das Feature `passkeys` steht im Starter-Kit mit `confirmPassword: true`. Basalto hat dasselbe mit denselben Versionen gebaut (Code-Login, Abweisung, Software-Authenticator für Tests).

Optionen

Option	Vorteile	Nachteile
A — Fortify nur für Passkeys, eigener Code-Login, Passwort-Endpunkte abweisen	Muster erprobt, Passkeys von Laravel	Fortify-Updates können Routen verschieben (Lektion)
B — Fortify ganz entfernen, Passkeys selbst	keine Überraschungen aus Fortify	WebAuthn selbst prüfen — mehr Risiko

Entscheid

Option A. `Fortify::authenticateUsing()` liefert `null`; Passwortbestätigung durch Bestätigung per Passkey bzw. Code ersetzt; Passwort-Spalten und -Tabellen entfernt; erstes Konto per `phpdbadmin:install`, weitere per Einladung.

Konsequenzen

- Test ruft jede alte Adresse des Starter-Kits auf (404/405).
- Vor jedem `composer update` Routenliste sichern und vergleichen.
- Parameter nie `{passkey}` nennen (globale Bindung von `laravel/passkeys`).

Nachtrag TB 00 (28.10.2026)

Umgesetzt mit einer strengeren Form von Option A: `Fortify::ignoreRoutes()` statt `authenticateUsing()`. Fortify registriert damit **keine** eigenen Routen; `routes/auth.php` trägt die Passkey-Wege der Bibliothek (`Laravel\Passkeys\Http\Controllers`) unter den Namen, die Fortify verwendet hätte. Ein Passwort-Endpunkt existiert gar nicht (`POST /login` → 405). Die Middleware

`password.confirm` leitet auf eine eigene Seite (Bestätigen per Passkey oder Code an die eigene Adresse); jede Anmeldung setzt die Bestätigung. Fortify bleibt nur, weil es die Passkey-Konfiguration (`passkeys.*`) aus `config/fortify.php` ableitet.

Ersteinrichtung (28.10.2026, Wunsch Hoang): Das erste Konto entsteht im Browser oder per Konsole. Im Browser schützt ein Einrichtungsschlüssel, den die App beim ersten Aufruf von `/setup` als Datei auf dem Server ablegt (`storage/app/private/einrichtungsschluessel.txt`, 0600) und nach der Einrichtung löscht — so wird nicht der erste Besucher einer frisch installierten Seite Administrator (T-19). Bewusst nicht gewählt: offene Einrichtung für den ersten Besucher (Übernahme), Freigabe nach Absender-IP (hinter einem Reverse-Proxy kommt jede Anfrage von `127.0.0.1`), Bestätigung per Mail (auf einem frischen Server oft noch nicht eingerichtet). Nach der Einrichtung folgt das Onboarding mit Passkey.

Siehe Solution-Architektur · Feasibility

Anhang I — ADR-009 — App-Datenbank: SQLite als Vorgabe

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Foundation; R-31)
Betrifft	Installation, Betrieb

Kontext

Die App-Datenbank hält Konten, Verbindungen, Verlauf und Protokoll — wenig Daten, wenige gleichzeitige Schreiber. Eine Installation neben fremden Datenbanken soll ohne zusätzlichen Server laufen.

Optionen

Option	Vorteile	Nachteile
A — SQLite als Vorgabe, die anderen drei möglich	keine Einrichtung, Sicherung = eine Datei	Schreibsperren bei vielen Parallelzugriffen (`busy_timeout`, WAL)
B — MariaDB/PostgreSQL vorgeschrieben	robuster bei vielen Konten	zusätzlicher Server nur für phpdbadmin

Entscheid

Option A, mit WAL und `busy_timeout`; die Testsuite läuft mit jeder der vier als App-Datenbank (B-06).

Konsequenzen

- Regeln für Spaltentypen gelten trotzdem für alle vier (Datenmodell § 1).
- Die App-Datenbank erscheint nie als Zielverbindung (R-17).

Siehe Solution-Architektur · Feasibility

Anhang J — ADR-010 — Lizenz MIT

Feld	Wert
Status	Angenommen
Datum	28.10.2026 (Entscheid Hoang, R-40; F-9)
Betrifft	Repository, Abhängigkeiten

Kontext

Hoang hat MIT gewählt (Markt überwiegend kostenlos, Schaufenster für Nguyen Development). Geprüft: Composer 101 Pakete MIT/BSD/Apache, `nette/*` wahlweise BSD-3; npm-Laufzeit MIT/ISC/BSD/Apache; neue Pakete MIT.

Optionen

Option	Vorteile	Nachteile
A — MIT	einfach, verbreitet	kein Schutz gegen geschlossene Ableitungen
B — GPL-3	Ableitungen bleiben offen	schreckt Einsatz in Firmen ab

Entscheid

Option A. `LICENSE` mit MIT vor dem ersten öffentlichen Push; Code aus GPL-Projekten (phpMyAdmin, phpPgAdmin, Adminer) wird nicht übernommen.

Konsequenzen

- Lizenzliste vor jedem Release erneut erzeugen.
- Hinweis «nicht verbunden mit phpMyAdmin» in README und Website (F-10).

Siehe Solution-Architektur · Feasibility

Anhang K — ADR-011 — PHP 8.5 als Mindestversion

Feld	Wert
Status	Angenommen (Hoang, 28.10.2026)
Datum	28.10.2026 (Foundation)
Betrifft	Installation, Hosting, `composer.json`

Kontext

Der Schutz von SQLite gegen `ATTACH` fremder Dateien braucht `Pdo\Sqlite::setAuthorizer()`. Geprüft in Herd: PHP 8.4.27 hat `Pdo\Sqlite`, aber keinen Authorizer; PHP 8.5.12 hat ihn. Laravel 13 verlangt nur PHP 8.3, `composer.json` steht auf `^8.3`.

Optionen

Option	Vorteile	Nachteile
A — PHP 8.5 vorschreiben	Schutz vollständig, ein Weg	Hosting mit PHP 8.4 fällt weg
B — PHP 8.4 erlauben, SQLite-`ATTACH` dort per Textprüfung sperren	breiter einsetzbar	Textprüfung ist schwächer; zwei Wege testen
C — PHP 8.4 erlauben, SQLite-Verbindungen dort nur lesend und ohne Editor	einfach	halbe Funktion

Entscheid

Option A (entschieden von Hoang am 28.10.2026). `composer.json` auf `^8.5` in TB 00; Herd läuft bereits mit 8.5.

Konsequenzen

- Die Installationsanleitung nennt PHP 8.5 und `pdo_sqlite`, `pdo_mysql`, `pdo_pgsql`.

Siehe Solution-Architektur · Feasibility

Anhang L — ADR-012 — SSH-Tunnel über das System-ssh

Feld	Wert
Status	Angenommen
Datum	29.10.2026 (TB 11, offener Punkt der Solution-Architektur § 16)
Betrifft	ConnectionFactory, SshTunnels, SshCommand, HostPolicy, ConnectionTester

Kontext

Datenbanken auf Servern sind oft nur über SSH erreichbar (V-08). phpdbadmin baut Verbindungen je Anfrage (ADR-002) und prüft jeden Host gegen die erlaubte Liste (V-05, T-05). Ein Tunnel darf diese Prüfung nicht umgehen, Schlüssel dürfen nicht im Klartext auf der Platte liegen, und kein Hilfsprozess darf die Anfrage überleben. Neue Abhängigkeiten sind nicht erwünscht.

Optionen

Option	Vorteile	Nachteile
A — PHP-Erweiterung `ssh2`	in PHP, keine Prozesse	Erweiterung fehlt in Herd und bei den meisten Hostern; Weiterleitung nur als Stream, PDO kann ihn nicht nutzen
B — Paket `phpseclib`	reines PHP	neue Abhängigkeit; Weiterleitung bräuchte einen eigenen lokalen Socket-Server in PHP
C — `ssh -N -L` als Hilfsprozess	OpenSSH ist überall da, wo es Server gibt; Host-Schlüssel, Algorithmen und Agent-freie Anmeldung erprobt; PDO verbindet ganz gewöhnlich zu `127.0.0.1:<Port>`	Prozess je Tunnel; Meldungen nur als Text auf stderr; `proc_open` muss erlaubt sein

Entscheid

Option C. `SshTunnels` startet `ssh` über Symfony Process mit einer Argumentliste (nie über eine Shell): `-N -F /dev/null -L 127.0.0.1:<freier Port>:<Ziel>:<Port>`, `BatchMode=yes`, `ExitOnForwardFailure=yes`, `StrictHostKeyChecking=accept-new` gegen die eigene `storage/app/ssh/known_hosts` mit `HostKeyAlias` = Name des Servers, ohne Agent und ohne fremde Schlüssel, `LogLevel=INFO` (nur so meldet ssh ein unerreichbares Ziel hinter dem Tunnel). Der private Schlüssel liegt verschlüsselt in der App-Datenbank (oder als Datei in einem erlaubten Ordner) und für ssh nur als Kopie mit 0600, bis die Anmeldung steht; eine Passphrase entfernt `ssh-keygen -p` aus dieser Kopie, die Passphrase geht über die Standardeingabe. Bereit ist der Tunnel, wenn das lokale Ende Verbindungen annimmt (Zeitgrenze `PHPDBADMIN_SSH_TIMEOUT`).

Geprüft wird vor dem Start: der SSH-Server wie jeder Host, das Ziel aus Sicht des Tunnels nach denselben Regeln (`HostPolicy::resolveTunnelTarget`: ein erlaubter Name geht unverändert an den SSH-Server, sonst nur eine geprüfte Adresse).

Konsequenzen

- Ein Tunnel lebt so lange wie seine Verbindung (`ConnectionFactory::release`), mehrere Verbindungen einer Anfrage teilen ihn; am Ende der Anfrage (`terminating`) und notfalls beim Herunterfahren von PHP werden alle beendet.
- «Verbindung testen» zeigt den Tunnel als eigenen Schritt mit Fingerabdruck; Fehler werden aus der Meldung von ssh zugeordnet (Schlüssel abgelehnt, Host-Schlüssel geändert, nicht erreichbar, Zeitüberschreitung, Ziel hinter dem Tunnel).
- `TunnelFailed` ist eine Unterart von `ConnectionNotAllowed` — überall, wo eine Verbindung nicht zustande kommt, erscheint der Tunnel mit derselben Fehlerkarte.
- Ohne `proc_open` oder ohne OpenSSH gibt es keinen Tunnel (Meldung «SSH nicht verfügbar»); SQLite braucht nie einen.

Siehe Solution-Architektur · Protokoll TB 11

Anhang M — Timebox 00 — Fundament

Stand: 28.10.2026 · ✅ abgeschlossen — gestartet auf Hoangs «Ja, TB 00 starten», gebaut in fünf Scheiben nach dem Timebox-Plan § 2; während Scheibe 5 auf Hoangs Wunsch ergänzt um **Ersteinrichtung und Passkey-Onboarding**.

Ziel

Ein sauberes Fundament ohne Starter-Kit-Reste: Anmeldung ohne Passwort, Verbindungen zu vier Datenbanken anlegen und testen, eine Dialekt-Schicht mit Vertragstests — geprüft gegen MySQL 8.4, MariaDB 10.11, PostgreSQL 18 und SQLite.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Starter-Kit bereinigen	A-01, A-04, A-05, B-01	✅
2 Anmeldung	A-02, A-03, A-06, A-09	✅ (A-03: Handtest mit Touch ID offen, Hoang)
3 Verbindungen	V-01, V-02, V-03, V-05, B-06	✅
4 Dialekt-Schicht	B-02 (+ Grundlage V-04, V-10)	✅
5 Qualität und Abnahme	B-03, B-08	✅ lokal; Workflow noch nie auf GitHub gelaufen (kein Remote)
Zusatz: Ersteinrichtung	A-06, A-03	✅ (Wunsch Hoang während Scheibe 5)

Wireframe-Bereiche 1 (Anmeldung), 2 (Code), 3 (Verbindungen), 4 (Verbindung anlegen), 13 (Konto).

Was entstanden ist

Anmeldung ohne Passwort (ADR-008, Nachtrag). Fortify registriert keine eigenen Routen mehr (`Fortify::ignoreRoutes()`); `routes/auth.php` trägt nur die Passkey-Wege der Bibliothek. `POST /login` antwortet 405 — einen Passwort-Endpunkt gibt es nicht. Code per Mail: sechs Ziffern, HMAC-Prüfsummen für Code, Adresse und IP, zehn Minuten, einmalig (bedingtes Update gegen zwei gleichzeitige Einlösungen), Sperre nach fünf Fehlversuchen, benannte Limiter je Adresse und IP, gleiche Antwort für unbekannte Adressen, die Mail geht erst nach der Antwort hinaus (`defer`), die Sitzung wird neu erzeugt. Der Code steht bewusst nicht im Betreff (Sperrbildschirm). Die Passwortbestätigung ist ersetzt durch «Bestätigen» per Passkey oder Code an die eigene Adresse; jede Anmeldung gilt als Bestätigung. Passkeys anlegen, umbenennen, entfernen im Konto. Protokoll: Anmeldungen (mit Weg: Code, Passkey, Einladung, Einrichtung), Fehlversuche (Adresse nur als Prüfsumme), Abmelden, Passkey-Änderungen.

Ersteinrichtung und Onboarding. Ohne jedes Konto zeigt `/login` die Ersteinrichtung statt einer Anmeldung ohne Ausweg. **Im Browser** (Hauptweg, Hoangs zweiter Wunsch: «nicht jeder hat ein Terminal»): `/setup` legt einen Einrichtungsschlüssel als Datei auf dem Server ab (`storage/app/private/einrichtungsschluessel.txt`, 0600, nie überschrieben, nach der Einrichtung gelöscht); wer die Installation hochgeladen hat, liest ihn über Dateimanager, FTP oder Editor und gibt ihn mit Name und E-Mail ein — ein zufälliger Besucher kann so nicht Administrator werden (T-19). **Per Konsole** (Alternative): `php artisan phpdbadmin:install` legt genau ein Administratorkonto an und gibt

einen signierten Link aus (30 Minuten, gilt nur bis zur ersten Anmeldung; erst ein Knopf meldet an, damit Link-Vorschauen ihn nicht verbrauchen). Die erste Anmeldung gelingt so auch, wenn der Server noch keine Mail versenden kann; solange die Ersteinrichtung offen ist, stellt der Befehl einen neuen Link aus. Wer sich ohne Passkey anmeldet (Code, Einladung, Einrichtungslink), landet im Onboarding «Passkey jetzt anlegen»; das erste Konto sieht die Variante «Ersteinrichtung» mit den nächsten Schritten. A-06 ist in CR-001 Fassung 5 entsprechend angepasst.

Verbindungen. Tabellen `connections` (ULID, Passwort mit `encrypted`-Cast) und `connection_user` nach dem Datenmodell. Anlegen, bearbeiten, entfernen für vier Treiber; Produktion schreibgeschützt vorgegeben; Passwort nie im Browser (die Seite erfährt nur, ob eines gespeichert ist), leeres Feld behält das gespeicherte, «nicht speichern» möglich. «Verbindung testen» mit Prüfschritten (Host erlaubt → Server erreichbar → Anmeldung → Rechte des Kontos), Fehler in Klartext je Treiber, Meldung des Servers zum Aufklappen. `HostPolicy`: Namen, Platzhalter, Adressen, Netze; einmal auflösen und zur geprüften Adresse verbinden; SQLite nur in erlaubten Ordnern über `realpath()`, die App-Datenbank nie. `{connection}` löst nur sichtbare Verbindungen auf — fremde sind 404 (T-16).

Dialekt-Schicht (B-02). Vertrag `App\Dialects\Dialect` mit Basis über Laravels Introspektion (Schema immer ausdrücklich) und vier Umsetzungen: Konfiguration (UTC, Zeitlimit beim Verbinden, SSL, Schreibschutz beim Öffnen bzw. für die Sitzung), Version, Fehlerzuordnung, Schemas, Tabellen und Sichten mit Zeilenschätzung, Check-Constraints und Trigger, Zeitlimit je Anweisung, Schreibschutz-Wächter (eine Anweisung je Aufruf in einer Nur-lesen-Transaktion, immer zurückgerollt), Kontorechte. `ConnectionFactory`: keine Mehrfachanweisungen, kein `LOCAL INFILE`, SQLite-`ATTACH` nur in erlaubten Ordnern (Authorizer).

Gestaltung. Palette aus `werkzeuge/kontraste.py` im Stylesheet (hell und dunkel), Inter und JetBrains Mono beim Build heruntergeladen und mit ausgeliefert (G-02), Wortmarke (G-06), Umgebungszeichen ○ ◐ ● mit Wort (G-03).

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
A-01	alte Adressen 404/405, Spalten weg	`StarterKitRemovalTest`: 17 Adressen 404, `POST /login` 405, keine Spalten `password`, `two_factor_*`, keine Tabelle `password_reset_tokens`
A-02	Erfolg, Ablauf, Wiederverwendung, falscher Code, Drosselung, unbekannte Adresse, Sitzung neu	`LoginCodesTest` (6), `LoginControllerTest` (14) — Sitzungs-ID vor/nach über das Cookie
A-03	Zeremonie mit virtuellem Authenticator; einmal Touch ID	`PasskeyLoginControllerTest`, `PasskeyManagementTest`, `ConfirmControllerTest` mit dem Software-Authenticator (aus Basalto); **Touch ID von Hand: offen (Hoang)**
A-04	`/dashboard` 404, Anmeldung führt zur Übersicht	`StarterKitRemovalTest`; Anmeldung mit Passkey → Übersicht, ohne Passkey → Onboarding
A-05	`/settings/profile` 404, Kontomenü	Test + Browser-Durchlauf (Menü Sicherheit, Darstellung, Abmelden)
A-06	Einladung annehmen, abgelaufene ablehnen; Befehl legt genau ein Admin-Konto an; falscher Schlüssel legt kein Konto an	`InvitationControllerTest` (7), `InstallCommandTest` (5), `InviteCommandTest` (3), `SetupControllerTest` (5), `FirstSetupControllerTest` (6), `SetupKeyTest` (4); im Browser mit falschem und richtigem Schlüssel durchgespielt

Anforderung	Kriterium (CR-001)	Nachweis
A-09	Eintrag mit Zeit, Konto bzw. Adresse (gehasht), IP-Prüfsumme	Tests prüfen `auth.login`, `auth.failed` (ohne Klartext-Adresse), `auth.logout`
V-01	je Treiber ein Test gegen den echten Dienst; Fehler am Feld	`ConnectionTesterTest` (Herd-Dienste), `ConnectionControllerTest`
V-02	je Fehlerart ein Test mit erzwungenem Fehler	nicht erreichbar, Zugang verweigert (ohne Passwort in der Meldung), Datenbank fehlt — je Server; SSL (PostgreSQL), Host nicht erlaubt, Datei keine Datenbank
V-03	kein Passwort in Props und JSON; Spalte verschlüsselt	Übersicht, Bearbeiten, Test-Endpunkt; Rohwert ≠ Klartext
V-05	nicht erlaubter Host und Pfad (auch `..`, Verknüpfung)	`HostPolicyTest` (21 Fälle aus 5 Tests), Controller am Feld; im Browser `~/Development/../../../etc/hosts` abgewiesen
B-01	Routen und Seiten ohne Starter-Kit-Reste	Routenliste: nur eigene und Passkey-Wege; Seitenordner `auth`, `connections`, `account`, `onboarding`, `probe`
B-02	Vertragstest gegen alle vier Umsetzungen	`DialectContractTest`: 12 Tests, 32 Fälle über die Treiber — alle grün
B-03	lokales Skript fährt alle vier; Workflow sparsam	`composer ci:local`; Workflow mit `paths-ignore`, `concurrency`, `timeout-minutes`, Matrix wöchentlich — **noch nie auf GitHub gelaufen**
B-06	Suite läuft mit jeder App-Datenbank	`composer test:app-db`: grün mit SQLite, MariaDB, MySQL, PostgreSQL als App-Datenbank
B-08	Prüfkette in der Reihenfolge der CI	`vp check` → `vue-tsc` → Pint → Larastan (Stufe 7) → PHPUnit

Stand der Suite: 221 Tests, 209 bestanden, 12 übersprungen (die schweren Raster- und Export-Proben mit 1 Mio. Zeilen, bewusst nur auf Wunsch).

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
01 Anmeldung	Sprachwahl Deutsch · English fehlt	Englisch ist nicht Teil von TB 00
01 Anmeldung	neu: Ersteinrichtung, wenn es kein Konto gibt	Wunsch Hoang; noch nicht im Generator
02 Code	«Code abgelaufen» hat keine eigene Ansicht; eine Meldung für falsch und abgelaufen	verrät nichts über den Zustand eines Codes
02 Code	Hinweis ergänzt: «…sofern es dazu ein Konto gibt»	ehrlich bei gleicher Antwort für unbekannte Adressen
03 Verbindungen	ohne «Zuletzt geöffnet», Favoriten, Befehlspalette, Klick in den Arbeitsbildschirm	kommen mit ihren Timeboxes (V-06, TB 01, TB 11)
04 Verbindung anlegen	SSL ohne «Zertifikat prüfen»	verbunden wird zur geprüften Adresse (V-05); Prüfen des Zertifikats braucht eine eigene Anforderung
13 Konto	ohne Sitzungen und Dichte	A-08 (TB 04), G-04 mit dem Raster (TB 01)
13 Konto	Kontomenü ohne «Verwaltung»	TB 07
neu	Ersteinrichtung (Link) und Onboarding «Passkey jetzt anlegen»	Wunsch Hoang; Wireframes dazu noch nicht im Generator

Browser-Durchlauf

Gegen einen gebauten Stand in einem Herd-Workspace (der eingebaute Browser erreicht den Vite-Dev-Server nicht): Bereiche 1–4 und 13 bei 1440 und 375 px, hell und dunkel, Anmeldung per Code mit echter Mail in Herd, Verbindung zu MariaDB anlegen und testen, abgewiesener Pfad mit `..`, Ersteinrichtung auf einer Installation ohne Konto. Gefunden und behoben: Adressen brachen mitten im Wort, Codefeld zu klein, Hinweis ohne Mail-Symbol und Ablaufzeit, Kopf «Verbindung testen» brach um, **Warnsymbol fehlte wegen eines vergessenen Imports** (jetzt meldet `vue-tsc` unbekannte Komponenten), «· Konto» ohne Datenbank, SQLite-Platzhalter mit einem hier nicht erlaubten Ordner, leeres Passwort galt als «gespeichert».

Entscheide in der Timebox

Entscheid	Begründung
`Fortify::ignoreRoutes()` statt `authenticateUsing()`	kein Passwort-Endpunkt statt eines abweisenden (ADR-008, Nachtrag)
Route `login.code.show` statt `login.code.form`	Wayfinder hängt verschachtelte Routen an die Funktion; `form` überschrieb die Formular-Variante und machte das Formular zum GET
kein «angemeldet bleiben» (`remember_token` entfernt)	Datenmodell § 1; Sitzungsdauer genügt
Probe-Seiten bleiben lokal hinter der Anmeldung	Sichtprüfung des Editors durch Hoang offen; Raster (TB 01) und SQL-Editor (TB 05) ersetzen sie
Proben F-1, F-7 und Teile von F-2 → Vertragstests	Probe-Dateien entfernt; Abbrechen, DDL-Vorschau, Raster und Export bleiben Proben bis zu ihren Timeboxes
Lesekonto `phpdbadmin_lesen` und Datenbanken `phpdbadmin_test` , `phpdbadmin_app_test` auf den lokalen Herd-Servern	Vertragstest «warnt nicht» (V-10) und Suite mit App-Datenbank auf den Servern

Commits

`083e3c9` Scheiben 1+2 · `b04de27` Scheibe 3 · `ceab0e8` Scheibe 4 · `2bea65c` Scheibe 5 (Prüfskript, Workflow, Funde) · `95aa78c` Ersteinrichtung und Onboarding · `7336538` Protokoll · `bfb091c` symfony/mime · `fc75729` Ersteinrichtung im Browser.

Offen

- **Touch ID von Hand** (A-03): einmal einen Passkey auf dem Mac anlegen und damit anmelden — Hoang.
- **Workflow auf GitHub:** noch kein Remote; der erste Push zeigt, ob beide Aufträge laufen.
- ~~**Sicherheitshinweis** `symfony/mime` (CVE-2026-45070, mittel)~~ — erledigt 28.10.2026 auf Hoangs Freigabe: 8.1.7 → 8.1.10, `composer audit` ohne Hinweise, Prüfkette auf allen vier App-Datenbanken grün, Testmail über Herd mit Umlauten im Betreff angekommen.
- Wireframes für Ersteinrichtung und Onboarding im Generator nachziehen — in Arbeit auf einem eigenen Zweig; dort fehlt noch die Einrichtung im Browser mit Einrichtungsschlüssel (`fc75729`).
- Sichtprüfung der Gestaltung durch Hoang am eigenen Bildschirm (G-01 bis G-08).

Gelernt

- Textersetzungen per Skript immer mit Prüfung, ob die Stelle gefunden wurde — der Formatierer bricht Zeilen um, und eine stille Nicht-Ersetzung liess ein Symbol verschwinden (Lektion im Vault).
- `vue-tsc` meldet unbekannte Komponenten erst mit `checkUnknownComponents` (Lektion im Vault).
- Wayfinder: Routennamen nie auf `.form` enden lassen (Lektion im Vault).
- Der eingebaute Browser erreicht den Vite-Dev-Server nicht; für Sichtprüfungen einen Herd-Workspace mit Build nutzen und dessen `artisan dev` anhalten (Projekt-Memory).

Anhang N — Timebox 01 — Katalog und Navigator

Stand: 29.10.2026 · ✅ abgeschlossen bis auf V-06 (Favoriten, «zuletzt geöffnet»), das mit TB 07 im selben Zweig gebaut wird, weil beides an `connection_user` hängt. Gestartet auf Hoangs «nächste tb»; ab dem Abend auf seinen Auftrag «mache alle TB durch» ohne Rückfragen weitergeführt.

Ziel

Eine Verbindung öffnen und darin arbeiten können: Arbeitsbildschirm mit Navigator für alle vier Datenbanken, Struktur einer Tabelle in denselben Begriffen, Schreibschutz und Umgebung sichtbar, Konten mit zu vielen Rechten gewarnt — auf Deutsch und Englisch.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Arbeitsbildschirm öffnen	V-04, V-10	✅ Kopf mit Umgebung (Farbe, Zeichen, Wort), roter Streifen bei Produktion, Titel «PROD · …», Kontowarnung; Passwort je Sitzung, Fehlerkarte statt Fehlerseite
2 Navigator	O-01	✅ Datenbank → Schema → Gruppen (Tabellen, Sichten, Routinen, Trigger, Ereignisse, Sequenzen) mit Filter «/»; 1'000 Tabellen in einem Test
Zusatz «Alle Datenbanken anzeigen»	V-01	✅ Wunsch Hoang; Fehler aus seinem Durchlauf (Endlosschleife) behoben
3 Strukturansicht	S-01, O-04	✅ Wireframe 08 bei 1440 und 375 px
4 Favoriten, zuletzt geöffnet	V-06	⏳ im Zweig von TB 07
5 Deutsch und Englisch	N-06, O-06	✅ alle Texte über das Sprachsystem, Test über alle Schlüssel
6 Abnahme	N-04, O-04	✅ O-04 Bildschirmfotos; N-04 Untergrenzen in der Wochen-Matrix (noch nie gelaufen, kein Remote)

Wireframe-Bereiche 3 (Verbindungen), 5 (Arbeitsbildschirm), 8 (Struktur).

Was entstanden ist

Arbeitsbildschirm (`/c/{verbindung}`, `/c/{verbindung}/s/{schema}`). `TargetConnections` öffnet Zielverbindungen je Anfrage mit Zeitlimit (15 s) und Schreibschutz, wenn die Verbindung schreibgeschützt ist; jede lesende Anweisung läuft zusätzlich im Wächter des Dialekts. Fehler der Zielverbindung werden zur Fehlerkarte mit Ursache und nächstem Schritt, fehlende Passwörter («nicht speichern») fragt eine eigene Seite einmal je Sitzung ab. Fehler der eigenen App-Datenbank bleiben Fehler der App (früher als «Verbindung gescheitert» getarnt — im Durchlauf von TB 02 gefunden).

Alle Datenbanken (Wunsch Hoang). Ohne Datenbanknamen zeigt MySQL/MariaDB alle Datenbanken als Schemas, PostgreSQL öffnet `postgres` und bietet die übrigen Datenbanken im Navigator an (`?db=`). Der Fehler aus Hoangs Durchlauf («Maximum call stack size … Infinite recursion») lag in Laravels `DatabaseManager`: zwei Zielverbindungen mit gleichem Namen trennten sich gegenseitig, der Neuaufbau

traf das falsche Objekt. Zielverbindungen stehen seither in keinem globalen Register; der Neuverbinder erneuert genau dieses Objekt und richtet Schreibschutz, Authorizer und Zeitlimit wieder ein. Gegenprobe: Auf dem alten Stand hängen die neuen Tests in der Schleife.

Struktur (`…/o/{objekt}/structure`). `Structure` liest Spalten mit Typ im Original und einer **Art**, die in allen vier Datenbanken gleich heisst (`ColumnKind`: Zahl, Text, Auswahl, Wahrheitswert, Datum und Zeit, JSON, Binär, Kennung, Geometrie, Liste, Sonstiges), Indizes, Fremdschlüssel in beide Richtungen, Checks, Trigger und DDL. Gleichmacher für O-04: MariaDB-JSON (Alias für `longtext` mit `json_valid`) gilt als JSON, eigene Aufzählungstypen in PostgreSQL als Auswahl, Vorgaben als SQL-Literal (`'offen'` statt `offen` bzw. `'offen'::character varying`), MariaDBs Text «NULL» als keine Vorgabe.

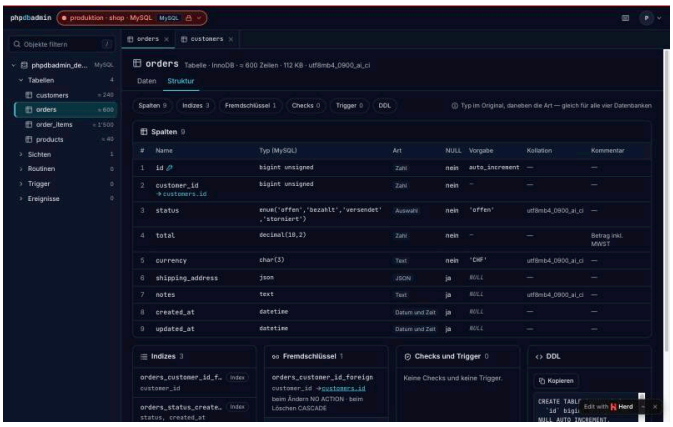
Deutsch und Englisch (N-06, O-06). Der deutsche Text ist der Schlüssel im Code (`__('…')`, `t('…')`, `tc('…')`), `lang/en.json` liefert Englisch (britisch). Sprache aus dem Konto, sonst aus dem Browser, sonst Deutsch; Mails in der Sprache des Kontos. Die Oberfläche bekommt die Übersetzungen einmal je Sprache (Once-Prop). Sprachwahl im Konto unter «Darstellung». Alle bestehenden Seiten hat ein Helfer in einer eigenen Arbeitskopie umgestellt (Zweig zusammengeführt, 322 Zeilen ohne Abweichung).

Abnahmekriterien

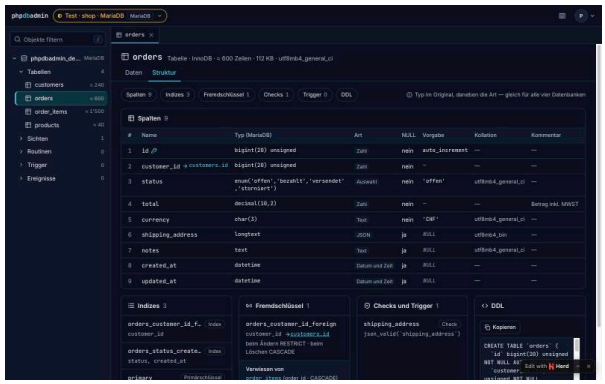
Anforderung	Kriterium (CR-001)	Nachweis
V-04	schreibgeschützte Verbindung lehnt jede schreibende Anweisung ab, auch Ausbruchsversuche	`DialectContractTest` (Wächter je Treiber), `TargetConnectionsTest` 19 Tests: Schreibschutz übersteht einen Neuaufbau (Laravel verbindet bei SQLSTATE 25006 neu)
V-10	Herd-`root` warnt, Lesekonto nicht	`DialectContractTest` gegen root und `phpdbadmin_lesen` auf MySQL, MariaDB, PostgreSQL
O-01	1'000 Tabellen flüssig	`WorkspaceControllerTest::test_navigator_with_a_thousand_tables_stays_fast` (unter 1 s inkl. Seite)
O-04	dieselbe Tabelle in allen vier Datenbanken	Bildschirmfotos unten; `StructureTest` 17 Tests (Art, Vorgaben, Fremdschlüssel beidseitig je Treiber)
S-01	Beispieltabelle mit allen Merkmalen je Treiber	`StructureTest` (Kommentar, Aufzählung, JSON, Standardwerte, eindeutiger Index, Check, Trigger, Sicht), `ObjectControllerTest`
N-06 / O-06	Sprachdateien vollständig ohne Rückfall	`LanguageFilesTest` (5): jeder Schlüssel mit englischer Zeile, keine verwaisten, gleiche Platzhalter, keine dynamischen Schlüssel, Feldnamen beider Sprachen; `LocaleControllerTest` (5)
N-04	Untergrenzen	Wochen-Matrix «untergrenzen» (MariaDB 10.6, MySQL 8.0, PostgreSQL 14) — **noch nie gelaufen** (kein Remote); lokal nur aktuelle Versionen

O-04: dieselbe Tabelle `orders` in allen vier Datenbanken

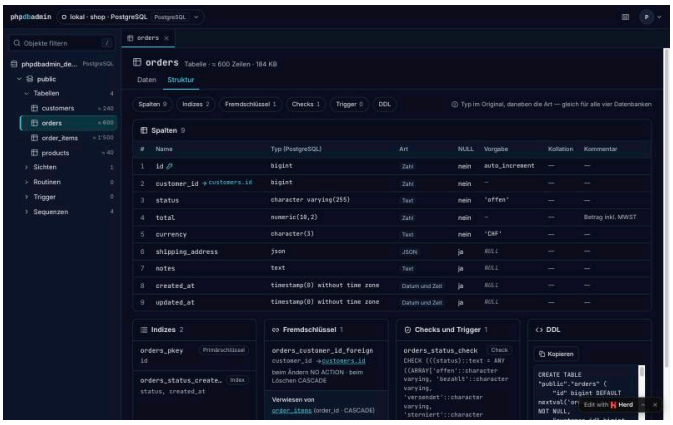
MySQL 8.4 (Produktion, schreibgeschützt)



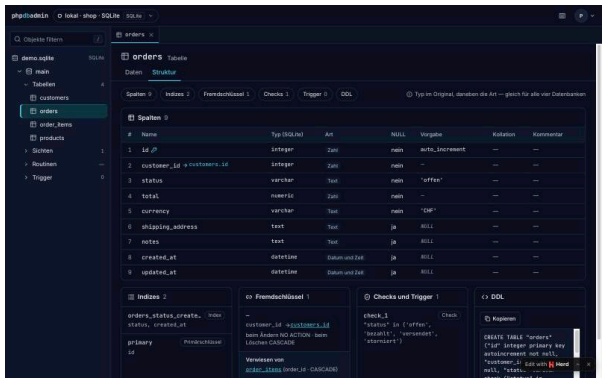
MariaDB 10.11 (Test)



PostgreSQL 18 (lokal)



SQLite (lokal)



Gleich: Aufbau, Begriffe, Art je Spalte, Vorgaben als Literal, Fremdschlüssel beidseitig. Erklärte Unterschiede: PostgreSQL legt Laravels `enum()` als `varchar` mit Check an (Art «Text», der Check steht daneben), SQLite kennt weder Zeilenschätzung noch Kollation noch Namen für Fremdschlüssel.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
08 Kopf	Grösse als eine Zahl statt «Daten · Indizes»	Laravel liefert die Gesamtgrösse; getrennte Werte bräuchten je Treiber eigene Abfragen — später, falls gewünscht
08 Knöpfe	«Tabelle ändern», «Exportieren», «Im Editor öffnen» fehlen	kommen mit TB 05, TB 06, TB 03
08 Reiter	«Beziehungen» und «SQL» fehlen	Beziehungsdiagramm S-07 (TB 10), SQL-Editor (TB 03)
05 Kopf	Suchfeld ⌘K fehlt	Befehlspalette O-02 (TB 03)

Tests

Neu oder erweitert: `StructureTest` (17), `ColumnKindTest` (25), `ObjectControllerTest` (Struktur-Teil), `TargetConnectionsTest` (+3, unabhängige Verbindungen, Schlüssel je Datenbank), `WorkspaceControllerTest` (+1, Ablauf aus Hoangs Durchlauf), `DialectContractTest` (+1, VACUUM trotz

ATTACH-Wächter), `LanguageFilesTest` (5), `LocaleControllerTest` (5) und die Englisch-Tests des Übersetzungs-Helfers. Zielserver-Tests laufen über `PHPDBADMIN_TEST_DATABASE` auch parallel in mehreren Arbeitskopien.

Commits

`d4766fa` Scheiben 1+2 · `0842694` Dialekt · `777ce66` alle Datenbanken · `a35ad37` Endlosschleife · `3dd2e9b` Struktur lesen · `884bf8e` Testdatenbank einstellbar · `dcf9e21` Sprachgrundlage · `6db5d35` Strukturansicht · `6a72d92` · `d2227f5` VACUUM · `558b4ae` Vorgaben, Chip · `c503ced` – `7f4c944` Sprache (Helfer) · `80fed1d` Merge · `8069824` Untergrenzen · `1c8f969` MariaDB-Vorgabe.

Gelernt

- Laravels `DB::build()` mit gleichem Namen ersetzt die vorhandene Verbindung; `DB::reconnect()` erneuert das Objekt im Register, nicht das in der Hand. Zielverbindungen deshalb ausserhalb des Registers bauen (Lektion im Vault).
- «Connection refused» hält Laravel für eine verlorene Verbindung und versucht neu — ohne Neuverbinder ein `LostConnectionException`. Sofort verbinden, dann kommt der Fehler als PDO-Fehler.
- Ein leeres PHP-Array wird zu JSON `[]`, und `[].sort` ist in JavaScript eine Methode: Zuordnungen an die Oberfläche als Objekt senden.
- Symfony schickt in Tests `Accept-Language: en-us` mit — mit Sprachwahl aus dem Browser kippen sonst alle deutschen Erwartungen.

Anhang O — Timebox 02 — Datenraster und Filter

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut.

Ziel

Die Daten jeder Tabelle und Sicht schnell und in allen vier Datenbanken gleich sehen: blättern, sortieren, filtern, suchen, zählen, Fremdschlüsseln folgen — auch bei einer Million Zeilen.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Datendienst	D-01, D-02, D-03, D-06	✅ `TableData`, `RowDetail`, `Cells` — 52 Tests über vier Treiber
2 Raster und Zeilen-Detail	D-01–D-03, D-06, O-09	✅ Wireframes 05 und 07 (Detail lesend; Bearbeiten mit TB 04)
3 Reiter und Kürzel	O-03, O-05	✅
Messung	N-01	✅ unter 20 ms für die erste Seite

Was entstanden ist

Datendienst. Blättern über den Primärschlüssel (zusammengesetzt ausgeschrieben, weil nicht jede Version Zeilenwerte vergleicht), sonst über den Versatz; der Primärschlüssel ist immer letztes Sortierkriterium. **NULL steht in beiden Richtungen zuletzt** — ausdrücklich über `case when … is null`, nicht nach Gewohnheit des Treibers. Geführte Filter (=, ≠, <, ≤, >, ≥, enthält, enthält nicht, beginnt, endet, ist NULL, ist nicht NULL) mit und/oder; alle Werte gebunden, `%` und `_` im Wert als Zeichen (Escape `!`). «enthält» ohne Gross-/Kleinschreibung: PostgreSQL `ILIKE` auf den Text der Spalte, MySQL/MariaDB nach Kollation (meist auch ohne Akzente), SQLite über eine Unicode-Faltfunktion (`phpdbadmin_fold`), die die `ConnectionFactory` registriert — sonst fände «MÜLLER» nicht «Müller». Freies WHERE als Klammerausdruck im Schreibschutz-Wächter mit Zeitlimit; Suche über alle Textspalten. Zählen genau bis `PHPDBADMIN_EXACT_COUNT_LIMIT` (50'000), darüber die Schätzung des Servers, gefiltert auf grossen Tabellen offen mit «genau zählen». Werte gleich in allen vier: Wahrheitswerte `true`/`false` (MySQL liefert 1/0), Binärdaten als Grösse und Anfang, lange Texte gekürzt, Ganzzahlen über 2^53 als Text, PostgreSQL-`bytea` aus dem Strom gelesen. Fremdschlüssel bekommen die Beschriftung der Zielzeile (bevorzugt `name`, `title`, `email` …). `TrustedSql` kennzeichnet die zwei Stellen mit absichtlich rohem SQL.

Raster (`/c/{verbindung}/s/{schema}/o/{objekt}`). Erste Seite kommt mit der Seite, weitere beim Scrollen (`rows`), dazu `count`, `row` und `view`. Virtualisiert (TanStack Virtual), Sortieren per Klick (Umschalt fügt hinzu), Breite ziehen, Spalten ausblenden — beides je Konto gemerkt (`table_views`). Tastatur:

Pfeile, Pos1/Ende, Bild auf/ab, Eingabe öffnet die Zeile. Filterleiste mit Chips, freiem WHERE, Suche und Zeilenzahl. Zeilen-Detail als Seitenblatt mit «verweist auf … öffnen» und «wird verwiesen von … 3 Zeilen zeigen». Telefon: Karten mit «alle Felder».

Zustände (O-09). Leere Tabelle, kein Treffer mit «Filter zurücksetzen», abgelehntes WHERE mit Meldung des Servers, Zeitlimit, abgerissene Verbindung und abgelaufene Sitzung beim Nachladen — je mit Ursache und nächstem Schritt.

Reiter (O-03). Jede geöffnete Tabelle oder Sicht wird ein Reiter mit ihrer letzten Adresse samt Filtern; gespeichert im Browser je Verbindung, übersteht das Neuladen. **Kürzel (O-05):** Übersicht über «?» oder das Tastatur-Symbol.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
D-01	Tests je Treiber mit Sortierung inkl. NULL, Reihenfolge ausdrücklich	`TableDataTest`: NULL zuletzt in beiden Richtungen, absteigend = Umkehrung von aufsteigend, Blättern ohne Lücken und Doppel (einfacher und zusammengesetzter Schlüssel, ohne Schlüssel), stabiler Versatz
D-02	je Treiber, mit Umlauten und Werten, die wie SQL aussehen	`x' or '1'='1`, `'; drop table people; --`, `%`, `_`, «müller»/«MÜLLER»/«jürg», und/oder, NULL, freies WHERE inkl. `1 = 1; delete from people` (nichts gelöscht)
D-03	Test je Treiber	Beschriftung des Fremdschlüssels, verweisende Zeilen gezählt, Zeile mit Verweisen in beide Richtungen
D-06	Test mit Schwelle	geschätzt über der Grenze, genau darunter, gefiltert offen, «genau zählen»; SQLite zählt mangels Schätzung immer genau
N-01	Messung mit erzeugten Daten je Treiber	`GridPerformanceTest` (`PHPDBADMIN_LEISTUNG=1`), Zahlen unten
O-03	Durchlauf	im Browser: zwei Reiter, Neuladen, beide da
O-05	Durchlauf nur mit Tastatur	Raster mit Pfeilen, Eingabe, Escape; Kürzel-Dialog über das Symbol. Die Taste «?» selbst liess sich mit dem Browser-Werkzeug nicht auslösen (es erzeugt kein `keydown`) — **von Hand offen**
O-09	leere Tabelle, SQL-Fehler, getrennte Verbindung, abgelaufene Sitzung	`ObjectControllerTest` (ungültiges WHERE → Fehlerkarte, 401/404/422/503 der JSON-Antworten), Oberfläche erklärt jede Ursache

N-01: erste Seite einer Tabelle mit einer Million Zeilen (lokal, Herd)

Fall	MySQL 8.4	MariaDB 10.11	PostgreSQL 18	SQLite
ohne Sortierung	6,5 ms	6 ms	17,1 ms	6,6 ms
`created_at` absteigend (Index)	1,5 ms	2,3 ms	2,4 ms	5,9 ms
Filter `category = 42` (ohne Index)	4,5 ms	4,5 ms	20,3 ms	35,5 ms
Suche über Textspalten	188,5 ms	138 ms	75,4 ms	156,7 ms

Zeit im Dienst inklusive Spalten, Zählung bzw. Schätzung und Beschriftungen.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
05 Kopf	«Zeile einfügen», «Exportieren», Änderungsleiste, «Rückgängig machen» fehlen	Bearbeiten TB 04, Export TB 06
05 Navigator	«Gespeichert» und «Verlauf» fehlen	SQL-Editor TB 03
07 Detail	lesend; «Duplizieren», «Zum Löschen markieren», «Zeile kopieren als» fehlen	TB 04 (D-04, E-05)
05 Raster	Spaltenkopf mit Typ statt Kurzform «decimal(10,2)» gekürzt	Breite der Spalte; der volle Typ steht in der Struktur

Tests

`TableDataTest` (52 = 13 × 4 Treiber), `ObjectControllerTest` (13), `GridPerformanceTest` (4, nur auf Wunsch). Suite: 393 Tests, 381 bestanden, 12 übersprungen (schwere Proben).

Commits

`88a95d7` Datendienst · `b2842e7` Raster und Detail · `88dec69` Reiter und Kürzel · `8af0231` Messung N-01. Die Raster-Probe der Feasibility (`/probe/raster`) ist entfernt.

Gelernt

- `rsync -R --delete` löscht auch in den Elternordnern des relativen Pfads — im Prüf-Workspace verschwanden so `database/database.sqlite`, Factories und Seeder (nur dort; Lektion im Vault).
- Larastan verlangt für rohes SQL ein Literal; statt die Prüfung zu umgehen, markiert `TrustedSql` die bewussten Stellen sichtbar.
- PostgreSQL schneidet als Text gebundene Binärdaten am Nullbyte ab — Testdaten für `bytea` als Literal setzen.

Anhang P — Timebox 03 — SQL-Editor und Befehlspalette → Alpha

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut. Q-07 (gefährliche Anweisungen, eigentlich TB 04) ist mitgekommen, weil der Editor es braucht.

Ziel

SQL schreiben und ausführen wie in einem Desktop-Werkzeug — mit Grenzen, die eine Datenbank in Produktion schützen, und Abbrechen, das wirklich abbricht.

Umfang

Scheibe	Anforderungen	Ergebnis
1 Zerlegen, einordnen, ausführen	Q-02, Q-03, Q-07, Q-09	✅ `StatementSplitter`, `StatementClassifier`, `QueryRunner`, Arbeitsprozess für SQLite
2 Editor, Verlauf, Palette	Q-01, Q-04, O-02	✅ Wireframes 10 und 12

Was entstanden ist

Zerleger. Eine Anweisung je Aufruf (die Verbindungen nehmen nie mehrere an, F-7). Kennt Zeichenketten mit den Escapes des Treibers (Backslash bei MySQL/MariaDB, `E'…'` bei PostgreSQL), Bezeichner (`"…"`, `` `…` ``, `[…]` bei SQLite), Kommentare (`--`, `#` bei MySQL, verschachtelte Blockkommentare bei PostgreSQL), Dollar-Blöcke, `BEGIN … END` in Routinen und Triggern (`END IF/LOOP/WHILE/REPEAT/CASE` richtig gezählt) und `DELIMITER` aus MySQL-Dumps; findet die Anweisung an der Schreibmarke (Byte-Position, der Browser rechnet UTF-16 in UTF-8 um).

Einordnen. Lesend (SELECT ohne INTO, SHOW, EXPLAIN ohne ANALYZE einer Änderung, WITH ohne ändernde Teile, lesendes PRAGMA …) oder schreibend; gefährlich sind DROP, TRUNCATE, DELETE und UPDATE ohne WHERE auf oberster Ebene und ALTER … DROP — Kommentare, Zeichenketten und Unterabfragen zählen nicht.

Ausführen. Lesendes immer im Schreibschutz-Wächter, Schreibendes nur mit Recht «schreiben» auf beschreibbaren Verbindungen und mit Protokolleintrag ohne Werte. Zeitgrenze `PHPDBADMIN_EDITOR_TIMEOUT` (30 s), Zeilen je Ergebnis `PHPDBADMIN_EDITOR_ROWS` (1'000, «mehr laden» bis 10'000). MySQL/MariaDB lesen ungepuffert und verwerfen den Rest mit `KILL QUERY`, PostgreSQL liest über einen Server-Cursor — nie liegt ein ganzes grosses Ergebnis im Speicher. **Abbrechen** ist eine zweite Anfrage: Ein Merker im Cache entscheidet über «abgebrochen» (MySQL kehrt nach `KILL QUERY` auch ohne Fehler zurück, F-2), die laufende Anweisung wird über eine eigene Verbindung beendet. **SQLite (Q-09)** läuft in einem Arbeitsprozess (`phpdbadmin:sqlite-worker`), den Zeitgrenze oder Abbruch beenden; fertige Ergebnisse davor bleiben.

Editor (Wireframe 10). CodeMirror 6 mit dem Dialekt des Treibers, Vervollständigung aus dem Schema (eine Abfrage je Treiber, als Deferred-Prop nachgeladen), Hervorhebung mit den Farben des Stylesheets, Formatieren ohne neue Abhängigkeit. ⌘↵ Anweisung an der Schreibmarke oder Auswahl, ⇧⌘↵ alles.

Ergebnisse als Reiter im virtualisierten Raster, als TSV kopieren. Gefährliches fragt nach, in Produktion mit dem Namen der Verbindung. Abfragen sind Reiter wie Tabellen, ihr Text bleibt im Browser; der Objektkopf hat den Reiter «SQL» mit einer Vorlage.

Verlauf und gespeicherte Abfragen (Q-04). `query_history` je Konto und Verbindung mit Suche (Frist 90 Tage, täglicher `model:prune`), `saved_queries` mit Namen.

Befehlspalette (O-02, Wireframe 12). ⌘K oder Suchfeld im Kopf: Objekte der offenen Verbindung, zuletzt Geöffnetes, andere Verbindungen (Suchquelle `/palette`, nur sichtbare), Aktionen («Neue Abfrage auf …», «Struktur von …», Server, Tastenkürzel); «>» zeigt nur Aktionen; Pfeile, Eingabe, Escape.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
Q-01	Durchlauf je Treiber	im Browser mit PostgreSQL: Vorlage, Formatieren, Ausführen, Vervollständigung (Endpunkt-Test: Spalten der Tabellen); Hervorhebung je Dialekt über `@codemirror/lang-sql`
Q-02	Abbrechen beendet die Abfrage auf dem Server; «abgebrochen» auch bei MySQL ohne Fehler	`QueryRunnerTest`: zweiter Prozess mit `sleep(20)` / `pg_sleep(20)` endet nach `KILL QUERY` / `pg_cancel_backend` in unter 5 s (MySQL, MariaDB, PostgreSQL); Merker entscheidet über «abgebrochen» (vier Treiber)
Q-03	lange und schreibende Anweisung auf geschützter Verbindung je Treiber	Zeitgrenze 1 s auf vier Treibern (MySQL mit grossem Kreuzprodukt, weil `SLEEP` still abbricht); Löschen auf schreibgeschützter Verbindung abgewiesen; ohne Recht nicht ausgeführt
Q-04	Test	`SqlControllerTest`: Verlauf nur des eigenen Kontos, Suche, gespeicherte Abfragen fremd 404
Q-07	Varianten mit Kommentaren, Gross-/Kleinschreibung, mehreren Anweisungen	`StatementClassifierTest` (13 Fälle Gefahr, 15 Fälle Art), `SqlControllerTest`: ohne Bestätigung 409 und nichts geändert, Produktion mit falschem Namen 409
Q-09	Abfrage über der Grenze beendet, Oberfläche meldet es	SQLite-Arbeitsprozess mit Zeitgrenze 1 s und mit Abbruch-Merker
O-02	Test der Suchquelle; Durchlauf nur mit Tastatur	`PaletteControllerTest` (nur sichtbare Verbindungen, keine Geheimnisse); im Browser ⌘K, Tippen, Auswahl

Alpha-Kriterium «Hoang nutzt phpdbadmin eine Woche zum Nachsehen»: **offen (Hoang)**.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
10 Werkzeugleiste	Auswahl des Schemas als Liste (bei mehreren)	Wireframe zeigt eine Verbindung mit einer Datenbank
10 Ergebnis	«Ergebnis exportieren» fehlt	Export TB 06
12 Palette	«Andere Verbindungen» zeigt Verbindungen, nicht ihre Tabellen	Tabellen anderer Verbindungen hiessen, jede zu öffnen — Suchquelle bewusst schlank

Tests

`StatementSplitterTest` 22, `StatementClassifierTest` 28, `QueryRunnerTest` 28 (vier Treiber), `SqlControllerTest` 8, `PaletteControllerTest` 2. Suite nach TB 03: 611 Tests, 595 bestanden, 16 übersprungen (schwere Messungen).

Commits

`efde8fe` Zerlegen, einordnen, ausführen · `d91231a` Editor, Verlauf, Palette. Die Editor-Probe der Feasibility ist entfernt.

Gelernt

- Ein eigener Hilfsname `run()` in einem PHPUnit-Test überschreibt die finale Methode der Testklasse — Fatal Error beim Laden.
- PostgreSQL: Nach einem Fehler im Cursor ist die Transaktion abgebrochen; ein `close` im `finally` wirft dann einen neuen Fehler und überdeckt den eigentlichen (Zeitlimit wurde zu «Fehler»).
- Unter PHP-FPM zeigt `PHP_BINARY` auf FPM — für einen Arbeitsprozess die CLI suchen (Einstellung `PHPDBADMIN_PHP_BINARY`).
- Der eingebaute Browser ist zwischen parallelen Helfern geteilt; Durchläufe macht nur der Hauptstrang.

Anhang Q — Timebox 04 — Bearbeiten mit Vorschau und Rückgängig

Stand: 29.10.2026 · ✅ abgeschlossen — auf Hoangs Auftrag «mache alle TB durch» ohne Rückfragen gebaut. Q-07 (gefährliche Anweisungen) ist schon mit TB 03 gekommen.

Ziel

Daten ändern, ohne Angst vor Datenverlust: alles zuerst sehen, in einer Transaktion anwenden, rückgängig machen können.

Umfang

Scheibe	Anforderungen	Ergebnis
1 RowEditor	D-04, D-07, D-09, N-03	✅ Vorschau, eine Transaktion, Sicherung, Rückgängig
2 Endpunkte	D-04, D-09, E-05, V-11	✅ inkl. Freigabe auf Zeit und Kopieren
3 Oberfläche	D-04, D-05, E-05, V-11	✅ Wireframes 5, 6, 7

Was entstanden ist

RowEditor (ADR-003). Gesammelte Änderungen — einfügen, ändern, löschen über den Primärschlüssel — zuerst als Vorschau aus `pretend()` mit eingesetzten Werten (nur zur Anzeige; ausgeführt wird mit Bindungen) und den Zeilen, die `ON DELETE CASCADE` mitlöschen würde. Anwenden auf **allen vier Treibern in einer Transaktion**: jede Änderung muss genau eine Zeile treffen, sonst wird alles zurückgerollt (`ChangeConflict`). Vorher- und Nachher-Werte in `change_snapshots` (bis 1'000 Zeilen, 7 Tage; Binärdaten als Base64), ins Protokoll nur SQL ohne Werte. **Rückgängig** kehrt rückwärts in einer Transaktion um — nur, wenn die Zeilen noch so aussehen wie nach der Änderung, sonst bleibt alles. Tabellen ohne Primärschlüssel nur lesen (Rückfall D-07), ungültiges JSON abgewiesen.

Oberfläche. Raster: Doppelklick, Eingabe oder F2 bearbeitet; geänderte Zellen, neue und zum Löschen markierte Zeilen farbig; Änderungsleiste; Vorschau als Seitenblatt mit Verfahren, nummerierten Anweisungen, Kaskaden-Hinweis und Namenseingabe in Produktion; danach «Zuletzt angewendet · Rückgängig machen». Zeilen-Detail mit Feld-Editor je Art (D-05): NULL-Haken (NULL ist nicht leer), Wahrheitswert, enum/set als Auswahl, JSON geprüft und formatierbar, lange Texte im Textfeld; Duplizieren, Löschmarke. **Kopieren (E-05)** als TSV, Markdown oder INSERT — Zeile oder Ausschnitt, auf dem Server erzeugt (Anführungszeichen je Treiber).

Schreibschutz auf Zeit (V-11). Für 15 Minuten, nur mit Recht «schreiben», mit Pflichtgrund, Grund und Ende im Protokoll; im Kopf «Schreiben erlaubt bis … · wieder sperren». Die Freigabe liegt in der Sitzung, `TargetConnections` berücksichtigt sie überall (Bearbeiten, SQL-Editor, Struktur, Server).

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
D-04	je Treiber: nichts geschrieben vor «Anwenden»; Fehler in der zweiten Anweisung rollt die erste zurück; Vorschau aus `pretend()` mit Bindungen; Kaskaden gezählt	`RowEditorTest` (vier Treiber): Vorschau ändert nichts, zählt 2 Kinder; doppelter Name in der zweiten Anweisung → erste zurückgerollt, kein Protokolleintrag
N-03	erzwungener Fehler mitten in einer Folge; Eltern mit Kindern	obiger Fall; verfehlte Zeile (Schlüssel 999) → nichts geändert; Kaskade löscht genau die Kinder des gelöschten Kunden
D-05 ↓	je Typ und Treiber byte-genau	Umlaute, Emoji, Anführungszeichen, Backslash, leer gegen NULL, Dezimal, Datum, Wahrheitswert, JSON auf vier Treibern; Binärdaten nur NULL (Rückfall: Bildvorschau später)
D-07 ↓	mehrdeutige Zeile wird nicht geändert	Tabelle ohne Schlüssel mit zwei gleichen Zeilen: abgewiesen, beide unverändert (Rückfall: nur lesen)
D-09	anwenden, rückgängig, Zustand gleich	vier Treiber: Ändern + Einfügen + Löschen, dann Rückgängig → Tabellen gleich dem Ausgangszustand; zweites Rückgängig abgewiesen; spätere fremde Änderung → Konflikt, nichts überschrieben
E-05	Test	`RowCopyTest` (Sonderzeichen, NULL, Binärdaten, Anführungszeichen je Treiber), `RowChangeControllerTest` (Zeile, Ausschnitt, Filter)
V-11	Ablauf nach 15 Minuten (Uhr gestellt), Grund und Ende im Protokoll, ohne Recht 403	`RowChangeControllerTest`: freigegeben → änderbar; nach 16 Minuten 403; Protokoll mit Grund und Ende; Leser 403; Sperren protokolliert
Q-07	—	mit TB 03

Durchlauf im Browser (MariaDB, Umgebung Test): Zelle ändern, Wahrheitswert umschalten, Vorschau, Anwenden, Rückgängig — Ausgangszustand wieder da.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
7 Detail	Binärdaten: keine Vorschau, kein Herunterladen, kein Ersetzen	Rückfall D-05: Werte als Text, Bildvorschau später
5 Kopf	«Exportieren» fehlt	TB 06
6 Vorschau	Hinweis «nicht in einer Transaktion» bei MyISAM fehlt	alle Testtabellen InnoDB; MyISAM ist selten und kennt keine Transaktion — als offene Lücke vermerkt

Tests

`RowEditorTest` 32 (vier Treiber), `RowChangeControllerTest` 8, `RowCopyTest` 3. Suite nach TB 04: 744 Tests, 728 bestanden, 16 übersprungen.

Commits

`7a82450` RowEditor · `fcf1d18` Endpunkte · `1066404` Oberfläche.

Gelernt

- `Model::create()` hat die Spaltenvorgaben der Datenbank nicht — eine Zeitzone fehlte (`setTimezone(null)`). Vorgaben gehören auch ins Modell (`$attributes`).
- MySQL quotet in `substituteBindingsIntoRawSql()` mit Backslash, die anderen mit verdoppeltem Hochkomma — Tests auf beides auslegen.

Anhang R — Timebox 06 — Export und Import im Hintergrund, Statistik

Stand: 29.10.2026 · ✅ gebaut auf einem eigenen Zweig (parallel zu TB 02/03/07), Sichtprüfung im Browser nach dem Zusammenführen durch den Hauptstrang.

Ziel

Tabellen und ganze Datenbanken als CSV, JSON oder SQL exportieren und CSV- oder SQL-Dateien importieren — in Paketen, im Hintergrund, mit Probelauf, der nichts schreibt, und aktueller Statistik danach; gleich auf allen vier Treibern.

Umfang

Anforderung	Priorität im Plan	Ergebnis
E-01 Export (Tabelle, Datenbank; CSV, JSON, SQL)	Must	✅ — Export einer **Abfrage** folgt mit dem SQL-Editor (TB 03), «gefilterte Zeilen» mit dem Knopf im Objektkopf
E-02 Import (CSV mit Zuordnung und Probelauf, SQL-Datei)	Must ↓	✅ beides, SQL-Datei samt Zerleger
E-06 Statistik nach Importen und auf Knopfdruck	Should	✅
B-07 Warteschlange, ohne Worker synchron mit Grenze	Should	✅

Wireframe-Bereich 11 (Export und Import).

Was entstanden ist

Export (`App\Services\Transfer\Exporter`, ADR-007). Gelesen wird im Schreibschutz-Wächter (`Dialect::runReadOnly`, eine Nur-lesen-Transaktion für den ganzen Export) und in Paketen zu 5'000 Zeilen über den Primärschlüssel — auch zusammengesetzt, `(a, b) > (x, y)` ausgeschrieben. Ohne Schlüssel und für Sichten liest ein Cursor, der nie das ganze Ergebnis hält: PostgreSQL `DECLARE … CURSOR` / `FETCH 5000`, MySQL und MariaDB ungepuffert, SQLite streamt ohnehin. Einen Versatz (`OFFSET`) gibt es nicht — ohne eindeutige Sortierung liefert er Zeilen doppelt oder gar nicht. Geschrieben wird als Strom nach `storage/app/private/transfers/<Auftrag>/`.

- **CSV:** Trennzeichen nach Wahl, UTF-8 mit BOM für Excel, Kopfzeile; NULL ist ein leeres Feld ohne Anführungszeichen, leerer Text `""` — so kommen beide beim Re-Import zurück. Binärspalten als Base64 oder weggelassen. CSV nur für eine Tabelle.
- **JSON:** Liste von Objekten, eine Zeile je Objekt; ganze Datenbank als Objekt «Tabelle → Liste». Zahlen und Wahrheitswerte als solche, JSON-Spalten eingebettet (MySQL, PostgreSQL), Binärdaten als Base64.
- **SQL:** Struktur, Daten oder beides, wahlweise mit `DROP … IF EXISTS`. `INSERT` zu 100 Zeilen bzw. 1 MB, Werte über Laravels `escape` je Treiber (`x'…'` bzw. `'\x…'::bytea`, `true`/`1`). Tabellen nach

Fremdschlüsseln geordnet; MySQL/MariaDB und SQLite ohne Schema im Namen (Import in eine andere Datenbank) und mit abgeschalteter Fremdschlüsselprüfung; PostgreSQL mit Schema wie die DDL des Dialekts, Sequenzen der `serial`-Spalten vorher, `setval` nach den Daten, Fremdschlüssel am Ende per `ALTER TABLE` (Verweise auf die eigene Tabelle und Kreise gehen so auch). MySQL-Sichten ohne `DEFINER` und ohne Schema vor den Spalten. Nicht enthalten: Trigger, Routinen, Ereignisse (SQLite-Trigger würden beim Einspielen feuern) — dafür E-03.
- Kommazahlen verlustfrei: `(string)` rundet in PHP auf 14 Stellen, auch in Laravels `escape`; der Export nimmt die kürzeste exakte Darstellung (`var_export`).

Import (`CsvImporter`, `SqlImporter`). Hochladen legt einen Entwurf an; danach Zuordnung, Probelauf, Import (Wireframe 11).

- **CSV:** Kopfzeile, Trennzeichen und Kodierung (UTF-8 mit/ohne BOM, sonst Windows-1252) erkennt die Analyse und lassen sich ändern; Zuordnung nach Namen vorgeschlagen (`E-Mail` → `email`). Eigener Leser statt `fgetcsv` (NULL gegen leeren Text, Zeilennummer des Satzanfangs auch bei Umbrüchen in Feldern). Eingefügt wird in Paketen zu höchstens 1'000 Zeilen (weniger bei vielen Spalten, unter den Grenzen für gebundene Werte); scheitert ein Paket, finden Einzelversuche hinter einem **Sicherungspunkt, der danach freigegeben wird**, die schuldigen Zeilen — Laravels verschachtelte Transaktionen geben ihre Sicherungspunkte nie frei, und PostgreSQL bricht ohne sie die ganze Transaktion ab. Fehler mit Zeilennummer und Grund (leer und NOT NULL, kein Base64, falsche Feldzahl, Ablehnung der Datenbank mit ihrer Meldung ohne SQL). Wahrheitswerte aus Tabellenkalkulationen (`ja`, `WAHR`, `x`) → 1/0. Leere Pflichtfelder mit Vorgabe oder Autowert setzen `DEFAULT` (SQLite kennt das nicht). Binärwerte als Strom gebunden (`PDO::PARAM_LOB`) — als Text verlöre PostgreSQL am Nullbyte und SQLite speicherte TEXT.
- **Probelauf schreibt nichts:** alle Zeilen in einer Transaktion einfügen, dann zurückrollen; auf Tabellen ohne Transaktionen (MyISAM) nur die Werte prüfen. Import: «Fehlerzeilen überspringen» oder «bei einem Fehler nichts importieren» — beides in einer Transaktion.
- **SQL-Datei:** eigener Zerleger `SqlScript` in Stücken von 1 MB — Zeichenketten (`''`, bei MySQL/MariaDB `\'`, bei PostgreSQL `E'…'`), Namen in `"…"`, `` `…` ``, `[…]`, Kommentare (`--` mit Leerraum bei MySQL, `#`, verschachtelt bei PostgreSQL), ausführbare `/*! … */`, `$$`-Blöcke, `DELIMITER` des MySQL-Clients, `BEGIN … END` in `CREATE TRIGGER/PROCEDURE/FUNCTION/EVENT`. Jede Anweisung einzeln vorbereitet (T-07); was MySQL nicht vorbereiten kann (Fehler 1295: `LOCK TABLES`, `CREATE TRIGGER` …), läuft ohne Vorbereitung — die Verbindung lässt nie mehrere Anweisungen in einem Aufruf zu. PostgreSQL und SQLite in einer Transaktion (eigene `BEGIN`/`COMMIT` der Datei übergangen, SQLite mit Fremdschlüsselprüfung am Ende); MySQL/MariaDB Anweisung für Anweisung, bei einem Fehler bleibt, was davor lief — die Meldung sagt es. Probelauf: nur zerlegen und zählen, Warnung vor `DROP`, `TRUNCATE`, `DELETE`/`UPDATE` ohne `WHERE`.
- **Schreiben nur mit Recht:** `Gate::authorize('write', $connection)` (vorläufige `DatabaseConnectionPolicy::write`: Besitzer oder «verwalten»; die vollständige Policy aus TB 07 ersetzt sie beim Zusammenführen), nie über schreibgeschützte Verbindungen (403), auf Produktion beim echten Lauf mit dem Namen der Verbindung. Jeder echte Import ein Eintrag `data.import` (Ziel, Zeilen, übersprungene, Statistik); eine SQL-Datei, die auf MySQL/MariaDB mittendrin scheitert, ebenfalls (mit `failed`).

Statistik (E-06). `Dialect::analyze`: `ANALYZE TABLE` bei MySQL/MariaDB (meldet Fehler als Ergebniszeile, deshalb gelesen und geprüft; aktualisiert auch die bis zu 24 Stunden alte Zahl in `information_schema` von MySQL 8), sonst `ANALYZE schema.tabelle`. Nach jedem CSV-Import automatisch (abwählbar), auf der Seite auf Knopfdruck mit Eintrag `data.analyze`.

Aufträge und Warteschlange (B-07). Tabelle `transfers` (Datenmodell § 2.4, ULID, Frist 24 Stunden, täglicher `model:prune` löscht Auftrag und Ordner). `RunExport` / `RunImport` tragen **nur die Kennung**; der Worker liest Zugangsdaten aus der App-Datenbank. Im Hintergrund läuft ein Auftrag, wenn ein Worker konfiguriert ist (`QUEUE_CONNECTION` nicht `sync`, `PHPDBADMIN_TRANSFERS_BACKGROUND` nicht `false`) **und** die Verbindung ihr Passwort speichert — «nur für diese Sitzung» kommt nie in die Warteschlange. Sonst läuft er in der Anfrage bis zu Grenzen (Vorgabe 100'000 Zeilen bzw. 20 MB) mit einer Meldung, die den Grund und den Ausweg nennt. Wer zuerst «wartet» auf «läuft» setzt, führt aus (bedingtes Update gegen Doppelläufe); `retry_after` der Warteschlange auf 3'660 s angehoben, weil ein Auftrag bis zu einer Stunde laufen darf. Fortschritt in `progress` und `rows`, höchstens zweimal je Sekunde geschrieben; dabei fragt der Lauf, ob abgebrochen wurde. Zeitlimit je Anweisung während eines Auftrags 600 s statt 15 s.

Oberfläche. Seite `workspace/transfers/Index` unter `c/{connection}/transfers` (Einstieg im Menü des Verbindungs-Chips): Exportieren, Importieren, Statistik, Aufträge mit Fortschritt, Abbrechen, Herunterladen bis zur Frist und Hinweis, wenn eine Minute lang kein Worker abholt. Die Oberfläche fragt den Stand alle 1,5 s über `transfers/status` ab (nur App-Datenbank) — und nur, solange ein Auftrag läuft. `?schema=…&table=…` belegt den Export vor (`transfersHref` in `resources/js/lib/transfers.ts`) — die Adresse für den Knopf «Exportieren» im Objektkopf. Seite `workspace/transfers/Import`: Schritte, Datei mit «anders lesen», Zuordnung mit Beispielwerten und Hinweis auf fehlende Pflichtspalten, Probelauf mit Fehlerzeilen, Import.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
E-01	Export und Re-Import ergeben dieselben Daten je Treiber	`RoundTripTest`: SQL (ganze Datenbank: Struktur und Daten → leere Datenbank) und CSV (Tabelle) je Treiber; Werte mit Umlauten, Emoji, `'`, `"`, `\`, `;`, Zeilenumbrüchen, `$$`, Text wie SQL, leerem Text gegen NULL, Nullbyte-Binärdaten, `0.30000000000000004`, `-1.0E-12`, Wahrheitswerten, JSON; Fremdschlüssel auf die eigene Tabelle, zusammengesetzter Schlüssel, Tabelle ohne Schlüssel, Sicht; nach dem Import zählt der Schlüssel weiter (PostgreSQL `setval`)
E-01	1 Mio. Zeilen unter 64 MB	`LargeTransferTest` (Gruppe `transfer-large`, nur mit `PROBE_GROSS=1`) — Zahlen unten
E-02	Tests mit Fehlerzeilen; Probelauf schreibt nichts	`CsvImporterTest` je Treiber: vier Fehlerzeilen mit Nummer 5–8 (auch nach einem Feld über zwei Zeilen) und Grund, Tabelle danach leer; überspringen, nichts importieren, Werte wie SQL wörtlich; `SqlImporterTest`: Probelauf führt nichts aus, Fehler nennt Anweisung und Zeile, PostgreSQL/SQLite rollen zurück; `ImportControllerTest`: 403 für schreibgeschützte Verbindung und Lesemitglied, Name der Verbindung auf Produktion, Zuordnung mit `id; drop table …` abgewiesen
E-06	Nach Import ist die Schätzung der Zeilenzahl aktuell	`StatisticsTest`: `Dialect::tables()` → `rows` = 500 ± 10 % auf MySQL, MariaDB, PostgreSQL; SQLite (keine Schätzung): `sqlite_stat1` = 500. Mutationsprobe ohne `ANALYZE`: MySQL (0), PostgreSQL (null), SQLite rot — **MariaDB bleibt grün**, InnoDB schätzt dort auch ohne
B-07	Test beider Wege	`TransferRunnerTest`: mit Worker in die Warteschlange (Auftrag trägt nur die Kennung) und vom Job ausgeführt; ohne Worker in der Anfrage; Passwort nur in der Sitzung → in der Anfrage trotz Worker, Grenze mit Erklärung; Job ohne gespeichertes Passwort scheitert mit Erklärung; Abbrechen während des Lesens; `TransferControllerTest`: Grenze ohne Worker, Warteschlange mit Worker

Messwerte

Eine Million Zeilen (`id`, Zahl, Text, Dezimalzahl, Zeitpunkt), je Treiber in einem eigenen Prozess gemessen (`LargeTransferTest`, 29.10.2026, MacBook mit Herd: MySQL 8.4, MariaDB 10.11, PostgreSQL 18, SQLite 3.45). Speicher: Spitze des PHP-Speichers über dem Stand davor; Prozess: Zuwachs von `ru_maxrss` während des Schritts (sieht auch libpq, F-8).

	MySQL	MariaDB	PostgreSQL	SQLite
Export CSV (49 MB)	5,0 s · PHP 4,1 MB · Prozess +4,4 MB	5,6 s · 4,1 MB · +4,5 MB	4,9 s · 3,8 MB · +4,5 MB	6,9 s · 3,6 MB · +5,8 MB
Export SQL, nur Daten (61 MB)	3,2 s · 3,2 MB · +0 MB	4,4 s · 3,2 MB · +0 MB	3,5 s · 3,0 MB · +0,2 MB	2,3 s · 2,8 MB · +0,1 MB
Import CSV (Pakete zu 1'000, mit ANALYZE)	9,0 s · 2,0 MB	6,1 s · 2,0 MB	9,2 s · 2,3 MB	3,6 s · 1,7 MB
Import SQL (10'000 INSERT zu 100 Zeilen)	11,3 s · 2,1 MB	5,9 s · 2,1 MB	16,5 s · 2,1 MB	2,8 s · 2,1 MB

Ergebnis: Abnahme E-01 (unter 64 MB) mit Abstand erfüllt — höchstens 4,1 MB PHP-Speicher und 5,8 MB Prozesswachstum; der Prozess wächst auch bei PostgreSQL nicht, weil Pakete über den Schlüssel libpq nie das ganze Ergebnis halten lassen. Der erste gemessene Schritt je Prozess trägt das Laden der Klassen mit.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
11 Export	eigene Seite je Verbindung statt Dialog über dem Raster; «gefilterte Zeilen» fehlt	der Knopf im Objektkopf und das Raster gehören dem Hauptstrang; er verlinkt mit `?schema=…&table=…` hierher
11 Export	ohne «noch etwa 20 s»	Fortschritt in Prozent und Zeilen; eine Restzeit wäre bei geschätzten Zeilenzahlen geraten
11 Import	Zuordnung nur nach gleichem Namen (nicht «Vorname» → `first_name`); kein Datumsformat «TT.MM.JJJJ»	Daten gehen wörtlich an die Datenbank; Formatumwandlung wäre eine eigene Anforderung
11 Import	Fehlerzeilen nicht als CSV zum Herunterladen	offen, siehe unten

Tests

Neu: `RoundTripTest` (16), `CsvImporterTest` (14), `SqlImporterTest` (10), `StatisticsTest` (4), `TableReaderTest` (8), `TransferRunnerTest` (8), `TransferTest` (1, Frist), `SqlScriptTest` (9, Zerleger), `TransferControllerTest` (11), `ImportControllerTest` (9), `LargeTransferTest` (4, nur auf Wunsch). Je Treiber gegen die Herd-Dienste: MySQL 12, MariaDB 14, PostgreSQL 13, SQLite 13 Fälle plus die HTTP-Tests auf SQLite; zweites Konto für fremde Aufträge (404) und Lesemitglied (403).

Stand der Suite: 480 Tests, 464 bestanden, 16 übersprungen (die schweren Proben mit 1 Mio. Zeilen, bewusst nur auf Wunsch); Pint, Larastan Stufe 7, `vp check` und `vue-tsc` ohne Befund.

Commits

`d466d6c` englische Zeilen des Rasters · `dd0dd7e` Scheibe 1: Server · `c7cf477` Scheibe 2: Oberfläche · Messtest und Protokoll (dieser Stand).

Offen

- Export einer **Abfrage** (mit dem SQL-Editor) und der **gefilterten Zeilen** des Rasters (Knopf «Exportieren» im Objektkopf).
- Fehlerzeilen eines Imports als CSV zum Korrigieren herunterladen (Wireframe 11).
- CSV-Export ohne Schutz vor Formeln (`=`, `+`, `-`, `@` am Anfang öffnet Excel als Formel) — bewusst, weil er die Daten verändern würde; Option für TB 08 (Sicherheitsprüfung).
- MySQL mit `sql_mode` `ANSI_QUOTES` oder `NO_BACKSLASH_ESCAPES`: der Zerleger erkennt das nicht.
- Sichtprüfung bei 1440 und 375 px im Browser — nach dem Zusammenführen.

Gelernt

- Laravel bindet Kommazahlen als Text mit 14 Stellen (`0.30000000000000004` → `0.3`), und `Connection::escape` tut dasselbe — für verlustfreie Werte `var_export` bzw. Werte aus der Datei als Text (Lektion im Vault).
- `Model::query()->value('status')` liefert mit Enum-Cast das Enum, nicht den Text; der Abbruch-Merker sah den Abbruch deshalb nie — erst ein Test mit echtem Abbruch zeigte es (`toBase()->value()`).
- MySQL 8 liefert Spalten aus `information_schema` in Grossbuchstaben (`ENGINE`) — immer mit Alias lesen.
- `pdf-bauen.py --help` kennt kein `--help` und baut alle sieben Dossiers neu (Lektion im Vault).

Anhang S — TB 07 — Konten, Rechte je Verbindung, Teilen, Protokoll

Stand: 29.10.2026 · ✅ gebaut auf einem eigenen Zweig (Basis `80fed1d`), noch nicht zusammengeführt — vorgezogen vor TB 04, parallel zu Raster und Export. Dazu V-06 (Favoriten, «zuletzt geöffnet») aus TB 01.

Ziel

Mehrere Personen arbeiten sicher auf einer Installation: Rechte je Verbindung, Teilen durch die Verwaltung, Konten verwalten im Browser und ein Protokoll, in dem nie ein Wert aus einer Zieldatenbank steht.

Umfang

Anforderung	Inhalt	Ergebnis
B-04 (Must)	Protokoll mit Verbindung, Ziel, SQL, Zeilen; Maskierung; Frist	✅ Dienst und Ansicht; schreibende Aktionen auf Zielverbindungen gibt es erst ab TB 04 — sie rufen `AuditLog::record()` mit `connection`, `target`, `sql`, `bindings`, `affectedRows`, `error`
A-07	Rollen und Rechte je Verbindung	✅ `DatabaseConnectionPolicy` (`view`, `write`, `manage`, `share`)
V-09	persönlich oder geteilt	✅ Dialog «Teilen …» und Rechte je Konto in der Verwaltung
A-05, A-06	Name und Adresse durch die Verwaltung; Einladung im Browser	✅
V-06	Favoriten, sortieren, «zuletzt geöffnet»	✅ (Suche und Gruppen gab es schon)

Wireframe-Bereiche 14 (Verwaltung: Konten, Einladungen, Protokoll) und 03 (Verbindungen: Favoriten, Sortierung).

Was entstanden ist

Rechte (A-07). Die Policy beantwortet vier Fähigkeiten; die Rechte bauen aufeinander auf (lesen ⊂ schreiben ⊂ verwalten):

Fähigkeit	erlaubt	wer
`view`	Daten und Struktur ansehen, SQL lesend, exportieren	jedes Recht
`write`	Daten und Struktur ändern, importieren, Schreibschutz auf Zeit aufheben (V-11)	«schreiben» und «verwalten»
`manage`	Verbindung bearbeiten, mit gespeichertem Passwort testen, entfernen	«verwalten»
`share`	Teilen und Rechte vergeben	nur Administratoren

Besitzer haben an ihren Verbindungen «verwalten», Administratoren an allen **geteilten**. Persönliche Verbindungen anderer sieht niemand, auch kein Administrator — sie bleiben 404 (Routenbindung, T-16); sichtbare ohne Recht antworten 403. `DatabaseConnection::permissionFor()` ist die eine Regel,

`scopeVisibleTo` und `isManageableBy` folgen ihr.

Teilen (V-09). In der Übersicht öffnet «Teilen …» (nur Administratoren) einen Dialog: Teilen ein/aus und je Konto «kein Zugriff · lesen · schreiben · verwalten». Ausschalten entzieht allen ausser dem Besitzer den Zugriff. Besitzer und Administratoren bekommen kein gespeichertes Recht. Karten zeigen «geteilt» und das eigene Recht.

Verwaltung (Wireframe 14). `/admin/accounts`, `/admin/invitations`, `/admin/audit`, nur Administratoren (Gate `administrate`, sonst 403), Einstieg im Kontomenü. Konten mit Rolle, letzter Anmeldung, Passkey; je Konto Name und Adresse ändern (A-05), Rolle, Sperren, Entfernen und «Rechte je Verbindung». `UserPolicy`: Rolle, Sperre und Entfernen nie am eigenen und nie am letzten aktiven Administratorkonto. **Gesperrt** (neue Spalte `users.disabled_at`): kein Code, kein Passkey-Zutritt, laufende Sitzungen enden mit der nächsten Anfrage (`EnsureAccountIsActive`), Sitzungen in der Datenbank werden gelöscht. **Entfernen:** Passkeys, persönliche Verbindungen und Rechte gehen mit; geteilte Verbindungen des Kontos übernimmt der Administrator. **Rolle verloren:** wer nicht mehr Administrator ist, startet ohne Rechte an fremden Verbindungen. **Einladungen** im Browser mit Ablauf, erneut senden, zurückziehen; nach dem Senden erscheint der Link einmal (für Installationen ohne Mail). `phpdbadmin:invite` nutzt denselben `InvitationSender`.

Protokoll (B-04). `audit_log` hat neu `connection_id` (ohne Fremdschlüssel — ein Eintrag überlebt seine Verbindung) sowie `user_name` und `connection_name` (lesbar nach dem Entfernen, Datenmodell § 4). Ansicht mit Filtern nach Konto, Verbindung, Art und Zeitraum, Suche in Anweisung und Ziel, Seiten zu 50, Export als CSV (Zellen mit `= + - @` entschärft; der Export steht selbst im Protokoll).

Maskierung (T-11). `App\Services\Audit\Masking`:

- `Masking::sql($sql, $bindings, $doubleQuotedStrings)` — Zeichenketten- und Zahl-Literale werden zu `?`; Bezeichner (`` `…` ``, `"…"`), Platzhalter (`?`, `$1`, `:name`) und Kommentare bleiben; `"…"` gilt bei MySQL und MariaDB als Zeichenkette.
- `Masking::message($message, $bindings)` — eingesetzte Bindungen (an Wortgrenzen, «1» trifft nicht «1062»), `'…'`, «near '…' at line», PostgreSQL «Key (…)=(…)», «Failing row contains (…)», «invalid input syntax … "…"».
- `Masking::exception($throwable)` — Meldung des Servers plus Anweisung mit Platzhaltern.

`AuditLog` speichert Anweisung und Fehler nur maskiert; Datenbankfehler gehen maskiert ins Log (Reporter in `bootstrap/app.php`, ersetzt den üblichen Eintrag); Zielverbindungen setzen `mask_bindings_in_exception_messages`.

Aufbewahrung. `phpdbadmin.audit.retention_days` (`PHPDBADMIN_AUDIT_RETENTION_DAYS`, Vorgabe 365, 0 = ohne Frist; Datenmodell § 5 entschieden: Konfiguration, angezeigt in der Verwaltung), `AuditEntry` ist `Prunable`, `model:prune` täglich im Scheduler (braucht den Cron-Eintrag für `schedule:run`).

Favoriten und «zuletzt geöffnet» (V-06). Stern je Karte, Filter «Favoriten», Favoriten oben; Sortierung umschaltbar (nach Gruppe dann zuletzt geöffnet · nach Name · zuletzt geöffnet zuerst), im Browser gemerkt. `WorkspaceController::show` setzt `last_opened_at`. Die Merker liegen in `connection_user`; Besitzer und Administratoren bekommen dafür eine Zeile mit ihrem wirksamen Recht «verwalten» — eine Merker-Zeile gibt so nie mehr Recht, als das Konto ohnehin hat.

Abnahmekriterien

Anforderung	Kriterium (CR-001)	Nachweis
A-07	Policy-Tests je Recht; fremde Verbindung 404 statt 403	`DatabaseConnectionPolicyTest` (10 Fälle: Rolle × Besitz × Teilen × Recht, dazu Sichtbarkeit = Policy), `ConnectionControllerTest` (403 für «lesen» auch vor der Validierung, 200 für «verwalten», Administrator 404 an persönlicher Verbindung)
V-09	persönliche Verbindung ist für andere 404	`SharingControllerTest` (9): Teilen, «kein Zugriff», Teilen beenden → 404 für Mitglied und Administrator, Besitzer weiter 200; nur Administratoren (403), Validierung
B-04	jede schreibende Aktion genau ein Eintrag; Bereinigung nach Frist; ein gebundener Wert erscheint in keiner Meldung	`AuditLogTest` (7): echte Fehler auf **MySQL, MariaDB, PostgreSQL, SQLite** (doppelter Schlüssel, falscher Zahlwert) — der Wert steht in keinem Feld des Eintrags; maskiertes Log; Frist 30 Tage und 0; `model:prune` täglich. `MaskingTest` (5 Tests, 14 Fälle). «Genau ein Eintrag je schreibender Aktion» prüfen die Aktionen ab TB 04
A-05	Name und E-Mail nur durch die Verwaltung	`AccountControllerTest` (13), `UserPolicyTest` (6 Fälle)
A-06	Einladung mit Ablauf	`Admin\InvitationControllerTest` (6), Rückzug macht den Link ungültig
V-06	Durchlauf	`FavoriteControllerTest` (6), `WorkspaceLastOpenedTest` (2); Browser-Durchlauf der Übersicht steht aus (siehe unten)

Gesperrte Konten: `LoginControllerTest`, `PasskeyLoginControllerTest`, `EnsureAccountIsActiveTest`.

Abweichungen von den Wireframes

Bereich	Abweichung	Grund
14 Legende «verwalten»	«zusätzlich Verbindung bearbeiten und entfernen» statt «… und mit anderen teilen»	Teilen ist nach V-09 und CR Sache der Administratoren
14 Konten	offene Einladungen stehen im Reiter «Einladungen», nicht zusätzlich in der Kontenliste; «Erneut senden» und «Zurückziehen» dort	eine Liste je Sache; der Reiter zählt die offenen
14 Konten	Rechte-Tabelle zeigt geteilte Verbindungen und die eigenen persönlichen des Administrators, keine persönlichen des gewählten Kontos	persönliche Verbindungen anderer sieht niemand (V-09)
14 Konten	«Entfernen» verlangt den Namen zur Bestätigung	G-08: zerstörende Handlung
14 Protokoll	«ältere laden» als Seiten (neuere · ältere)	einfache Seiten mit Filtern in der Adresse
14 Protokoll	Ergebnis «abgelehnt» für Fehlversuche, «Fehler» bei gescheiterten Aktionen; «schreibgeschützt» erst mit TB 04	es gibt noch keine schreibenden Aktionen
03 Verbindungen	Seitenleiste «Zuletzt geöffnet» (Tabellen, Abfragen) fehlt; die Karten zeigen «zuletzt geöffnet» je Verbindung	Verlauf je Objekt kommt mit Q-04 (TB 03)
03 Verbindungen	Favoriten oben in jedem Abschnitt, nicht in einem eigenen Abschnitt	wie im Wireframe (Stern an der Karte); Filter «Favoriten» zeigt nur sie

Die Wireframes 03 und 14 sind im Generator noch nicht nachgezogen.

Tests

Neu: 11 Testdateien mit 60 Tests (davon zwei Matrizen mit 16 Fällen); erweitert: `ConnectionControllerTest`, `LoginControllerTest`, `PasskeyLoginControllerTest`. Die Maskierung ist gegen echte Fehlermeldungen aller vier Treiber geprüft. Die neuen Tests laufen grün mit SQLite, MariaDB, MySQL und PostgreSQL als App-Datenbank; die Migrationen laufen auf allen vier. Zweites Konto für Rechte: in jedem Endpunkt-Test (Mitglied 403, fremde Verbindung 404, Gast zur Anmeldung).

Gesamtlauf zuletzt: 476 Tests, 464 bestanden, 12 übersprungen (die schweren Proben).

Entscheide in der Timebox

Entscheid	Begründung
Administratoren sehen alle geteilten, aber keine persönlichen Verbindungen anderer	«Administrator darf alles» ohne Einblick in persönliche Zugänge; Datenmodell § 2.2
Merker-Zeilen für Besitzer und Administratoren mit Recht «verwalten» statt `permission` leer erlauben	keine Spaltenänderung (der Migrations-Wächter lehnt `change()` ab); die Zeile gibt nie mehr Recht als die Rolle; beim Rollenverlust werden die Zeilen an fremden Verbindungen gelöscht
`connection_id` im Protokoll ohne Fremdschlüssel, mit Namen von Konto und Verbindung	Einträge überleben Konto und Verbindung (Datenmodell § 4)
Neue Migrationen ohne `down()`	Protokoll- und Kontodaten sollen beim Zurückrollen nicht still verschwinden; der Migrations-Wächter weist `dropColumn` ab
Einladungslink einmal auf der Seite	wie der Befehl; der Link steht nur bis zur nächsten Anfrage in der Sitzung
Datenbankfehler maskiert ins Log statt des üblichen Eintrags	Datenmodell § 1.9 und Solution-Architektur § 13: keine Werte aus Zieldatenbanken im Log

Commits

`4322f65` fehlende englische Zeilen der Basis · `f720d71` Scheibe 1 Policy · `863b47b` Scheibe 2 Teilen · `3359df0` Scheibe 3 Verwaltung · `6d2f991` Scheibe 4 Protokoll · `f7eac95` Scheibe 5 Favoriten · Doku.

Offen

- **Browser-Durchlauf** nach dem Zusammenführen (Hauptstrang): Übersicht mit Favoriten und Sortierung, Protokoll mit echten Einträgen, bei 1440 und 375 px, hell und dunkel. Teilen, Konten und Einladungen sind vor dem Zusammenführen im Workspace `tb07-rechte` durchgespielt.
- **Fehlermeldungen im Raster** (TB 02): `TableData` sollte Meldungen der Server mit `Masking::exception()` zeigen; Zielverbindungen maskieren schon die eingesetzten Bindungen, nicht aber die Werte in den Meldungen der Server.
- **Schreibende Aktionen** (TB 04 ff.) protokollieren mit `AuditLog::record(..., connection:, target:, sql:, bindings:, affectedRows:, error:)` und prüfen mit `write` / `manage` der Policy.
- Fristen für Anmeldecodes und Einladungen (`model:prune`, Datenmodell § 4) — der tägliche Lauf steht, die Modelle sind noch nicht `Prunable`.
- Wireframes 03 und 14 im Generator nachziehen.

Gelernt

- Der Migrations-Wächter (Plugin) weist `dropColumn` und `change()` ab — Schemaänderungen additiv planen (Spalte dazu, keine Typänderung).
- Merker und Rechte in derselben Tabelle: Eine Merker-Zeile darf nie ein Recht geben, das die Rolle nicht schon gibt — sonst bleibt beim Rollenwechsel still ein Zugriff zurück.
- `mask_bindings_in_exception_messages` maskiert nur Laravels eingesetzte Anweisung; MySQL und PostgreSQL zitieren Werte auch in ihrer eigenen Meldung («Duplicate entry '…'», «Key (…)=(…)»).

Anhang T — Timebox 09 — Server-Verwaltung

Stand: 29.10.2026 · ✅ gebaut und getestet in drei Scheiben auf einem eigenen Zweig (Arbeitskopie), vorgezogen vor v1.0 — Browser-Durchlauf und Zusammenführen im Hauptstrang offen.

Ziel

Den Server hinter einer Verbindung ansehen und geführt verwalten — Übersicht, Prozesse, Variablen, Konten und Rechte, Wartung —, mit derselben Bedienung für MySQL, MariaDB, PostgreSQL und SQLite und ohne dass etwas ungesehen passiert.

Umfang

Nr.	Anforderung (CR-001 § 4.8)	Prio	Ergebnis
W-01	Server-Übersicht: Version, Laufzeit, Grösse je Datenbank, Zeichensätze	S	✅
W-02	Prozesse und Sitzungen anzeigen und beenden	S	✅
W-03	Variablen und Status lesen	C	✅ (Durchlauf offen)
W-04	Benutzer, Rollen und Rechte geführt verwalten mit Vorschau	S	✅
W-05	Wartung: ANALYZE, VACUUM, OPTIMIZE, Integritätsprüfung	C	✅ (Durchlauf offen)

Kein eigenes Wireframe (Timebox-Plan: «in TB 09 entworfen»); gestaltet im Stil der Wireframes 05 (Arbeitsbildschirm), 06 (SQL-Vorschau) und 08 (Struktur).

Was entstanden ist

Dialekt (ADR-001). Der Vertrag kennt jetzt `serverOverview`, `sessionId`, `processes`, `endProcessStatement` / `endProcess`, `variables`, `statusValues`, `accounts`, `currentAccount`, `privilegeChoices`, `createAccountStatements`, `dropAccountStatements`, `privilegeStatements`, `maintenanceOperations`, `maintenanceForTables`, `maintenanceStatements` und `runMaintenance` — umgesetzt in allen vier Dialekten; SQLite gibt `null` bzw. leere Listen, wo es Prozesse und Konten nicht gibt. Was ändert, plant der Dialekt nur als `PlannedStatement` (echter Text und Text für Vorschau und Protokoll, Passwort maskiert).

Ändernde Aktionen. `TargetConnections::withWritable()` öffnet für genau eine Aktion eine Verbindung ohne Schreibschutz — nie für eine schreibgeschützte (`ReadOnlyConnection`), getrennt von den Leseverbindungen und danach sofort freigegeben. `ServerPlans` plant aus geprüften Eingaben; jeder Name eines Ziels muss im Katalog stehen. `ServerChanges` zeigt die Vorschau über die Leseverbindung und führt über die beschreibbare aus, Anweisung für Anweisung, ohne Abfrage-Ereignisse (dort stünde das Passwort); jede Aktion ergibt genau einen Eintrag im Protokoll — `server.kill`, `server.user.create`, `server.user.drop`, `server.grant`, `server.revoke`, `server.maintenance` —, gelungen oder gescheitert, mit SQL ohne Geheimnisse. `AuditLog::record()` nimmt dafür das SQL auf.

Seiten. Unter `c/{connection}/server` fünf Inertia-Seiten im Arbeitsbildschirm (`workspace/server/Overview`, `Processes`, `Variables`, `Accounts`, `Maintenance`), Einstieg «Server» im Menü der Verbindung:

Methode	Adresse	Name
GET	`server`	`workspace.server`
GET	`server/processes`	`workspace.server.processes`
POST	`server/processes/{process}/end`	`workspace.server.processes.end`
GET	`server/variables`	`workspace.server.variables`
GET	`server/accounts`	`workspace.server.accounts`
POST	`server/accounts/preview`	`workspace.server.accounts.preview` (JSON)
POST	`server/accounts`	`workspace.server.accounts.apply`
GET	`server/maintenance`	`workspace.server.maintenance`
POST	`server/maintenance/preview`	`workspace.server.maintenance.preview` (JSON)
POST	`server/maintenance`	`workspace.server.maintenance.run`

Abnahmekriterien

Anforderung	Kriterium	Nachweis
W-01	Test je Treiber	Vertrag: Version, Laufzeit (SQLite keine), Zeichensatz, Grösse der Test-Datenbank > 0, PostgreSQL zusätzlich je Schema, SQLite Seitengrösse, Journal und Datei — vier Treiber; Seite je Server-Treiber und SQLite
W-02	beendete Sitzung ist weg	Vertrag je Server-Treiber: zweite Sitzung öffnen, Kennung über `connection_id()` bzw. `pg_backend_pid()`, beenden — danach nicht mehr in der Liste und tot (eigene PDO-Verbindung scheitert), ein zweites Beenden meldet «gibt es nicht»; Abbrechen stoppt `sleep` in einer asynchronen Sitzung (mysqli, pgsql), die Sitzung bleibt. Endpunkt je Server-Treiber: Weiterleitung, Meldung, genau ein Eintrag `server.kill`. Die eigene Sitzung ist markiert und abgewiesen
W-03	Durchlauf	Vertrag vier Treiber, Seite für SQLite und PostgreSQL; Browser-Durchlauf offen
W-04	Tests je Treiber (ohne SQLite)	Vertrag je Server-Treiber: Konto mit einem Passwort, das wie SQL aussieht, anlegen, mit diesem Passwort anmelden, Recht auf eine Tabelle erteilen, mit `SHOW GRANTS` bzw. `has_table_privilege` und mit einem echten Zugriff prüfen, entziehen, löschen; Rechte auf Server, Datenbank und (PostgreSQL) Schema erscheinen in der Liste und verschwinden wieder. Endpunkt je Server-Treiber: Vorschau maskiert und ändert nichts, Lebenszyklus mit vier Einträgen im Protokoll ohne Passwort
W-05	Durchlauf	Vertrag je Treiber und Art (11 Fälle) für eine Tabelle und das ganze Schema ohne Fehler; SQLite erkennt einen beschädigten Index; Endpunkt mit Ergebnis je Tabelle, VACUUM auf SQLite trotz ATTACH-Wächter; Browser-Durchlauf offen

Querschnitt: ohne «verwalten» 403, schreibgeschützt 403 (auch für den Besitzer), fremde Verbindung 404, Produktion erst mit dem eingetippten Namen — für alle fünf ändernden Endpunkte, ohne Eintrag im Protokoll. Eingaben, die wie SQL aussehen (Kontoname, Host, Recht, Datenbank, Tabelle «*», Ebene, Prozesskennung), werden vor jeder Anweisung abgewiesen.

Gestaltungsentscheide (statt Wireframe-Abweichungen)

Bereich	Entscheid	Grund
Einstieg	Eintrag «Server» im Menü der Verbindung; Navigator unverändert, zeigt die Schemas ohne Auswahl	Navigator und Layout bearbeitet parallel jemand anderes

Bereich	Entscheid	Grund
Bereiche	Reiter wie in der Strukturansicht (Wireframe 08), brechen um statt zu scrollen	gleiche Bedienung wie im übrigen Arbeitsbildschirm
Bestätigung	Dialog im Stil der SQL-Vorschau (Wireframe 06): nummerierte Anweisungen, Hinweise je Treiber, auf Produktion Name der Verbindung, Fehler des Servers im Dialog	eine Bestätigung für Prozesse, Konten und Wartung
Konten	erst ein Formular (anlegen bzw. Ebene, Ziel, Rechte), dann die Vorschau; Löschen geht direkt zur Vorschau	nichts läuft ohne Vorschau, auch nicht «nur» ein REVOKE
Ebenen	MySQL/MariaDB: Server, Datenbank, Tabelle; PostgreSQL: Server (vordefinierte Rollen wie `pg_monitor`), Datenbank, Schema (USAGE/CREATE und Rechte auf allen Tabellen darin), Tabelle	PostgreSQL kennt keine Rechte auf «.»
Schmale Fenster	Prozesse, Konten und Variablen als Karten bzw. umbrechende Zeilen, keine breiten Tabellen	375 px lesbar (O-07)
Gesperrt	Knöpfe bleiben sichtbar, aber gesperrt, mit Grund (schreibgeschützt bzw. ohne «verwalten»)	man sieht, was ginge
SQLite	gleiche Reiter; Prozesse und Konten mit klarer Erklärung, Variablen = lesbare PRAGMAs, Status = Optionen der Bibliothek	gleiche Bildschirme für alle vier (O-04)

Entscheide in der Timebox

Entscheid	Begründung
Bezeichner je Segment selbst quoten (Anführungszeichen verdoppelt) statt `Grammar::wrap()`	`wrap()` trennt an Punkten und « as » und lässt «» *ungequotet — eine Tabelle* «» würde zu «alle Tabellen»
Zeichenketten (Passwort, Host, Kontoname bei MySQL) über `PDO::quote`, Rechte nur aus der festen Liste je Ebene	DDL wie `CREATE USER` nimmt keine Platzhalter
MySQL/MariaDB: `_` und `%` im Datenbanknamen eines GRANT maskiert	auf Datenbank-Ebene sind sie Platzhalter — «my_db» träfe auch «myXdb»
Vorschau als JSON (`useHttp`), Ausführen als Inertia-POST mit Weiterleitung und Meldung	dieselbe Planung für beide; die Seite lädt nach einer Änderung neu
Ein Eintrag im Protokoll auch für gescheiterte Versuche (`outcome: failed`, Meldung maskiert)	wer was versucht hat, gehört ins Protokoll
Meldungen des Servers aus `getPrevious()` der `QueryException`, Geheimnisse maskiert	Laravel hängt die Anweisung samt Werten an die Meldung
PostgreSQL meldet sich als `application_name = phpdbadmin`	eigene Sitzungen in `pg_stat_activity` erkennbar
PostgreSQL: `DROP ROLE` ohne `DROP OWNED`	`DROP OWNED` löscht Objekte; lieber ablehnen lassen und den Hinweis zeigen
Wartung mit eigenem Zeitlimit (`PHPDBADMIN_MAINTENANCE_TIMEOUT_MS`, Vorgabe 10 Minuten)	OPTIMIZE und VACUUM grosser Tabellen brauchen länger als eine Abfrage
SQLite-VACUUM: `ATTACH ''` im Authorizer erlaubt (Zeile aus dem Hauptstrang)	VACUUM hängt intern eine leere Datenbank an

Tests

126 neue Tests; Suite gesamt 449, davon 437 bestanden und 12 übersprungen (die schweren Proben auf Wunsch). Testläufe mit `PHPDBADMIN_TEST_DATABASE=phpdbadmin_test_server`; Konten heissen `phpdbadmin_w04_…` und werden im Test wieder entfernt (`finally`).

Datei	Tests	MySQL	MariaDB	PostgreSQL	SQLite
`Dialects/DialectServerContractTest`	46	12	15	11	8
`Http/Controllers/Workspace/Server/*` (fünf Dateien)	53	5	25	6	17
`Http/Requests/Workspace/Server/ServerChangeRequestTest`	18	—	—	—	18
`Services/Server/ServerPlansTest`	7	3	2	2	—
`Connections/TargetConnectionsTest` (neu: beschreibbare Verbindung)	2	—	—	—	2
zusammen	**126**	**20**	**42**	**19**	**45**

Prüfkette vor jedem Commit: `vp check`, `vue-tsc`, Pint, Larastan, PHPUnit — grün.

Commits

`5d54279` Scheibe 1: Dialekt für die Server-Verwaltung · `2272649` Scheibe 2: Grundlage für ändernde Aktionen · `87b5a20` Scheibe 3: Seiten und Endpunkte · Doku (dieser Commit).

Offen

- **Browser-Durchlauf** bei 1440, 768 und 375 px, hell und dunkel, gegen einen Herd-Workspace — Hauptstrang.
- **PostgreSQL schickt das Passwort im Klartext** (`CREATE ROLE … PASSWORD '…'`); mit `log_statement = ddl` steht es im Server-Log. Abhilfe wäre ein SCRAM-Verifier, im Client berechnet — PHP fehlt dafür SASLprep.
- **Rollen in MariaDB** werden gelistet, aber nicht gelöscht (`DROP ROLE`); MySQL-Rollen erscheinen als gesperrte Konten.
- **Anonyme Konten in MariaDB** (Herd: `''@'localhost'`) gehen `'app'@'%'` vor — ein neues Konto meldet sich lokal erst mit Host `localhost` an. Ein Hinweis in der Oberfläche fehlt.
- Integritätsprüfung für PostgreSQL (`amcheck`) nicht im Umfang; die Prozessliste aktualisiert sich nur auf Knopfdruck.
- Grosse Wartung kann an der Laufzeitgrenze von PHP scheitern, bevor das Zeitlimit des Servers greift.

Gelernt

- `Grammar::wrap()` ist kein Quoting für beliebige Namen (Punkte, « as », «*») — für Bezeichner aus Eingaben eigenes Quoting je Segment (Lektion für den Vault).
- MariaDB hängt an `SHOW GRANTS` den Passwort-Hash (`IDENTIFIED BY PASSWORD '*…'`) — vor der Anzeige entfernen.
- Inertia-Tests verlangen die Seitendatei: Endpunkte und Seiten gehören in dieselbe Scheibe.

- Pint (`php_unit_method_casing`) benennt private Hilfsmethoden, die mit «test» beginnen, in Testklassen in `test_…` um — Hilfsmethoden anders benennen.

Anhang U — Timebox 11 — Schemavergleich, Laravel-Migration, Herd-Erkennung, SSH-Tunnel

Stand: 29.10.2026 · ✅ gebaut und getestet in vier Scheiben auf einem eigenen Zweig (Arbeitskopie) — Browser-Durchlauf (auch auf Hoangs Mac mit Herd) und Zusammenführen im Hauptstrang offen.

Ziel

Komfort für Entwickler: zwei Schemas vergleichen und das Ziel per Skript angleichen — auch zwischen zwei Treibern —, aus einer Tabelle eine Laravel-Migration erzeugen, Verbindungen aus Herd vorschlagen und Server hinter SSH erreichen, ohne dass dabei Geheimnisse den Server verlassen oder die erlaubten Hosts umgangen werden.

Umfang

Nr.	Anforderung (CR-001 § 4)	Prio	Ergebnis
S-08	Schemavergleich zweier Datenbanken mit Änderungsskript	C	✅
S-09	Laravel-Migration aus einer bestehenden Tabelle erzeugen	C	✅
V-07	Herd-Dienste und `.env` der Herd-Sites als Vorschläge erkennen	C	✅ (Durchlauf auf Hoangs Mac offen; Erkennung gegen seinen Herd-Ordner gelesen: 3 Dienste, 30 Sites in 60 ms)
V-08	SSH-Tunnel	C	✅

Keine Wireframes (Stufe 4); gestaltet im Stil der Wireframes 04 (Verbindung anlegen), 06 (SQL-Vorschau) und 08 (Struktur).

Was entstanden ist

Gestalt eines Schemas (`app/Services/Shape`). `ShapeReader` liest Tabellen über Laravels Introspektion (Spalten, Indizes, Fremdschlüssel) und die Checks des Dialekts — mit denselben Regeln für die Art wie die Strukturansicht (JSON in MariaDB über `json_valid`, eigene Aufzählungen in PostgreSQL). Jede Spalte bekommt ihre Laravel-Spalte (`LaravelTypes`: `bigInteger` + `unsigned`, `string(100)`, `decimal(10, 2)`, `enum` auch aus den Checks, die Laravel in PostgreSQL und SQLite anlegt), Vorgaben werden vergleichbar gelesen (`DefaultValue`: MySQL ohne, MariaDB mit Anführungszeichen, PostgreSQL mit Typumwandlung, `current_timestamp()` = `now()` = `CURRENT_TIMESTAMP`).

Vergleich (S-08). `ShapeComparer` vergleicht Tabellen, Spalten (gleicher Treiber: Typ im Original; verschiedene Treiber: Art mit dem, was Laravel auf dem anderen Treiber daraus macht, dazu Länge, wo beide eine melden), NULL, Vorgabe, Autoinkrement, Indizes und Fremdschlüssel über Spalten und Ziel statt über Namen. `ChangeScriptBuilder` baut daraus das Skript im Dialekt des Ziels über `Blueprint::toSql()` der Zielverbindung; ausgeführt wird nichts. Entfernen, was nur im Ziel steht, nur auf Wunsch.

Migration (S-09). `MigrationWriter` schreibt `Schema::create` mit Kurzformen (`id()`, `timestamps()`, `softDeletes()`, `rememberToken()`), Fremdschlüssel mit Regeln, Indizes ohne Namen, wo Laravel denselben vergäbe; was Laravel nicht kennt, steht als Kommentar mit Hinweis darin.

Herd (V-07). `HerdDiscovery` liest `config/services.plist` (Dienste mit Port; ob einer läuft, zeigt ein offener Port), `config/valet/config.json` (geparkte Ordner und der Ordner der verknüpften Sites) und die `.env` jeder Site (`DB_*` oder `DB_URL`).

SSH (V-08, ADR-012). `SshTunnels` startet `ssh -N -L` über Symfony Process; `ConnectionFactory` verbindet zum lokalen Ende und gibt den Tunnel mit der Verbindung frei. `HostPolicy::resolveTunnelTarget` prüft das Ziel aus Sicht des Tunnels, `resolveSshKeyPath` Schlüsseldateien. `KnownHosts` verwaltet `storage/app/ssh/known_hosts` (Fingerabdruck, vergessen).

Seiten und Adressen.

Methode	Adresse	Name	Seite
GET	`compare`	`compare`	`compare/Index` — Quelle, Ziel, Liste, Skript (Kopieren, Herunterladen, im SQL-Editor des Ziels öffnen); Einstieg «Schemas vergleichen» in der Verbindungsübersicht
GET	`c/{connection}/s/{schema}/o/{object}/migration`	`workspace.object.migration`	`workspace/Migration` — Datei zum Kopieren und Herunterladen; Einstieg «Laravel-Migration» in der Strukturansicht
DELETE	`connections/{connection}/ssh-host-key`	`connections.ssh-host-key.destroy`	Host-Schlüssel vergessen (verwalten, Protokoll `connection.ssh_host_key_forgotten`)

Das Formular «Verbindung anlegen» zeigt Administratoren lokal die Vorschläge aus Herd (nachgeladen, mit Skelett) und hat für Server einen Abschnitt «SSH-Tunnel».

Abnahmekriterien

Anforderung	Kriterium	Nachweis
S-08	Test mit zwei Ständen	Je Treiber zwei Schemas auf demselben Server (zweite Datenbank `…_b`, in PostgreSQL Schema `vergleich_b`, in SQLite zweite Datei): alle Unterschiede gefunden (Tabelle neu und nur im Ziel, Länge, NULL, Vorgabe, Spalte neu und nur im Ziel, eindeutiger Index statt gewöhnlichem, Index neu, Fremdschlüssel mit anderer Regel); Skript angewendet → erneuter Vergleich ohne Unterschied; ohne «entfernen» bleibt genau, was nur im Ziel steht. **Gegenseitig zwischen Treibern:** MySQL → PostgreSQL, PostgreSQL → MySQL, SQLite → MariaDB, MariaDB → SQLite, PostgreSQL → SQLite, MySQL → MariaDB — Skript angewendet, danach keine Unterschiede in Tabellen, Spalten (Art), Indizes, Fremdschlüsseln
S-09	Migration erzeugt dieselbe Tabelle auf einem anderen Treiber	Generierte Datei in einen Ordner geschrieben, mit `include` geladen, `up()` gegen die Testdatenbank des anderen Treibers (MySQL → PostgreSQL, PostgreSQL → MySQL, SQLite → MariaDB, MariaDB → SQLite), danach mit dem Vergleich aus S-08 geprüft: drei Tabellen mit Fremdschlüssel ohne Unterschied

Anforderung	Kriterium	Nachweis
V-07	Durchlauf auf Hoangs Mac	Test gegen einen nachgebauten Herd-Ordner (Dienste, geparkte und verknüpfte Sites, `.env` mit `DB_URL`, SQLite, nicht erlaubter Host); Passwörter weder im Vorschlag noch in der nachgeladenen Antwort; Passwort nur für denselben Host, Port und Benutzer. Erkennung gegen Hoangs echten Herd-Ordner gelesen (ohne Passwörter auszugeben). **Browser-Durchlauf offen**
V-08	Test gegen einen lokalen SSH-Dienst	`SshTunnelTest` gegen `PHPDBADMIN_TEST_SSH_*`: Tunnel zu MariaDB mit Fingerabdruck, gespeicherte Verbindung öffnet lesend und schreibend über **einen** Tunnel und schliesst ihn am Ende, Schlüssel abgelehnt, Host-Schlüssel geändert, Datenbank hinter dem Tunnel nicht erreichbar. Auf diesem Mac läuft kein SSH-Dienst (Remote Login aus) — der Test überspringt dann sichtbar; für die Abnahme lief er gegen einen vorübergehend gestarteten `sshd` als Benutzer auf `127.0.0.1:22222` (nur Schlüssel), 10 von 10 bestanden. Ohne Dienst laufen immer: falsche und richtige Passphrase, nicht erlaubter SSH-Server, Ziel ausserhalb der Liste, Schlüsseldatei ausserhalb der Ordner

Querschnitt: Vergleich und Migration brauchen nur «ansehen» (es wird nichts ausgeführt), fremde Verbindungen sind 404 (T-16). Passwort, Schlüssel und Passphrase gehen nie an den Browser (Seiten, nachgeladene Eigenschaften, JSON von «Verbindung testen») und nie ins Protokoll — dort steht nur `ssh: true`. Schlüssel liegen verschlüsselt (Rohwert ≠ Klartext, Test).

Gestaltungsentscheide

Bereich	Entscheid	Grund
Vergleich	eigene Seite unter `compare`, Einstieg in der Übersicht; Quelle und Ziel nebeneinander, auf dem Telefon untereinander mit Tausch-Knopf	zwei Verbindungen passen nicht in den Arbeitsbildschirm einer Verbindung
Vergleichen	erst auf Knopfdruck (`run=1`), alles in der Adresse	die Wahl lädt nur die Listen; ein Vergleich lässt sich neu laden und teilen
Begriffe	«neu» (nur in der Quelle, wird angelegt), «nur im Ziel», «geändert» mit den Merkmalen (Typ, Länge, NULL, Vorgabe)	«fehlt» wäre je nach Blickrichtung doppeldeutig
Skript	Kopf mit Quelle, Ziel, Verfahren des Treibers (ADR-003); Hinweise über dem Skript; im SQL-Editor bis 4'000 Zeichen	die Adresse des Editors trägt die Vorlage; längere Skripte kopieren oder herunterladen
Migration	eigene Seite mit Kopf der Tabelle; in der Strukturansicht nur ein Link	Struktur bearbeitet parallel TB 05
Herd	Dienste zuerst, dann sechs Sites, «Alle zeigen»; nicht erlaubte Vorschläge sichtbar, aber gesperrt	keine Scrollbalken; man sieht, was ginge
SSH	Abschnitt im Formular, Schlüssel einfügen oder Datei auf dem Server; Fingerabdruck nach dem Test; «Host-Schlüssel vergessen» nur, wenn einer gespeichert ist	Fingerabdruck beim ersten Mal vergleichen können

Entscheide in der Timebox

Entscheid	Begründung
SSH über das System-ssh, nicht `ssh2` oder phpseclib (ADR-012)	keine neue Abhängigkeit; PDO verbindet gewöhnlich zum lokalen Ende
`HostKeyAlias` = Name des SSH-Servers, verbunden wird zur geprüften Adresse	DNS-Rebinding bleibt ausgeschlossen, und ein anderer Server unter demselben Namen fällt trotzdem auf

Entscheid	Begründung
Passphrase mit `ssh-keygen -p` über die Standardeingabe aus einer Kopie entfernen	`BatchMode` fragt nie; über die Befehlszeile stünde sie in `ps`
`LogLevel=INFO`	nur so meldet ssh «channel N: open failed», wenn die Datenbank hinter dem Tunnel fehlt
`TunnelFailed` als Unterart von `ConnectionNotAllowed`	alle Fehlerkarten (Arbeitsbildschirm, Raster, Editor, Server) zeigen den Tunnel ohne eigene Zweige
Ziel hinter dem Tunnel: erlaubter Name unverändert, sonst geprüfte Adresse	«nach denselben Regeln aus Sicht des Tunnels» — der Tunnel öffnet keinen Weg, den die Liste nicht erlaubt
Zwischen Treibern Art statt Typ vergleichen, Länge nur, wo beide eine kennen	SQLite speichert keine Länge, PostgreSQL kennt kein `unsigned`, `dateTime` und `timestamp` sind dort gleich
Spalten von Fremdschlüsseln bekommen den Ganzzahltyp ihres Ziels (Skript und Migration)	MySQL lehnt `bigint` → `bigint unsigned` ab; aus PostgreSQL und SQLite kommen Schlüssel ohne «unsigned»
Index, den MySQL für einen Fremdschlüssel selbst anlegt, zählt nicht	er fehlte sonst in jedem Vergleich gegen PostgreSQL und SQLite
Namen von Indizes und Schlüsseln zwischen Treibern neu vergeben (`tabelle_spalten_art`, höchstens 60 Zeichen)	in PostgreSQL und SQLite sind Indexnamen je Schema eindeutig, in MySQL je Tabelle
SQLite: alles einer Tabelle in einem Blueprint, Umbau-Befehle zusammen	sonst liest der zweite Umbau den alten Stand und stellt einen entfernten Fremdschlüssel wieder her
PostgreSQL: geänderte Aufzählung als Text ändern, Check bleibt	`ALTER COLUMN … TYPE` kann keinen Check anlegen
Herd nur bei `APP_ENV=local` und vorhandenem Ordner, nur Administratoren; Passwort nur für denselben Host, Port und Benutzer	fremde `.env` enthalten Geheimnisse anderer Projekte
Migration ohne `down()` für die SSH-Spalten	der Wächter `db-migration-guard` lässt kein Entfernen von Spalten zu

Tests

111 neue Tests; Suite gesamt 949, davon 924 bestanden und 25 übersprungen (20 wie bisher, dazu die 5 Tests gegen einen SSH-Dienst, der auf diesem Mac nicht läuft). Mit dem vorübergehenden SSH-Dienst bestand `SshTunnelTest` 10 von 10. Testläufe mit `PHPDBADMIN_TEST_DATABASE=phpdbadmin_test_vergleich`; zweite Datenbanken heissen `phpdbadmin_test_vergleich_b` bzw. Schema `vergleich_b` und werden im Test wieder entfernt.

Datei	Tests	MySQL	MariaDB	PostgreSQL	SQLite	ohne Datenbank
`Services/Shape/ShapeComparerTest` (je Treiber 3, sechs Treiberpaare)	18	3 + 3 Paare	3 + 3 Paare	3 + 3 Paare	3 + 3 Paare	—
`Services/Shape/MigrationWriterTest` (vier Paare, Methoden, Ausbruch)	6	1 + 2 Paare	2 Paare	2 Paare	2 Paare	1
`Http/Controllers/CompareControllerTest`	7	—	—	—	7	—
`Http/Controllers/Workspace/MigrationControllerTest`	3	—	—	—	3	—

Datei	Tests	MySQL	MariaDB	PostgreSQL	SQLite	ohne Datenbank
`Unit/Services/Shape/DefaultValueTest`	28	—	—	—	—	28
`Connections/SshTunnelTest` (5 gegen SSH-Dienst, 5 immer)	10	—	5	—	—	5
`Unit/Connections/SshCommandTest`, `SshTunnelsTest`	15	—	—	—	—	15
`Unit/Connections/HostPolicyTest` (neu: Ziel hinter dem Tunnel, Schlüsseldatei)	9	—	—	—	—	9
`Connections/HerdDiscoveryTest`	4	—	—	—	—	4
`Http/Controllers/Connections/ConnectionControllerTest` (neu: SSH 7, Herd 3)	10	—	—	—	—	10
`Http/Controllers/Connections/TestConnectionControllerTest` (neu: SSH)	1	—	—	—	—	1
zusammen	**111**					

Prüfkette vor jedem Commit: `vp check`, `vue-tsc`, Pint, Larastan, PHPUnit — grün.

SSH-Dienst für den Test (nicht im Repo; so lief die Abnahme): Host- und Client-Schlüssel mit `ssh-keygen`, `sshd_config` mit `Port 22222`, `ListenAddress 127.0.0.1`, eigener `AuthorizedKeysFile`, `PasswordAuthentication no`, `UsePAM no`, `AllowTcpForwarding local`; Start mit `/usr/sbin/sshd -D -e -f sshd_config` als Benutzer, danach `PHPDBADMIN_TEST_SSH_PORT=22222 PHPDBADMIN_TEST_SSH_KEY=<client_key> php artisan test --filter SshTunnelTest`.

Commits

`59cd1af` Scheibe 1: Schemavergleich mit Änderungsskript (S-08) · `c1d4fa8` Scheibe 2: Laravel-Migration (S-09) · `3c7fffc` Scheibe 3: SSH-Tunnel (V-08) · `ccb37c2` SSH: Datenbank hinter dem Tunnel · `998012a` Scheibe 4: Herd-Erkennung (V-07) · `f1af885` SSH: Schlüsseldatei bleibt beim Bearbeiten · Doku (dieser Commit).

Offen

- **Browser-Durchlauf** bei 1440, 768 und 375 px, hell und dunkel — Vergleich, Migration, Formular mit Herd-Vorschlägen und SSH-Abschnitt — Hauptstrang; V-07 auf Hoangs Mac.
- **Vergleich unter der Verbindung:** die Seite nimmt `?source=<Verbindung>` schon an; ein Einstieg aus dem Arbeitsbildschirm fehlt (Navigator und Kopf gehören TB 10a bzw. dem Hauptstrang).
- **Nicht übertragbar** und als Hinweis im Skript: Checks ausser Aufzählung und JSON, eigene Typen in PostgreSQL (Aufzählungen als Text), Ausdrücke als Vorgabe zwischen Treibern (ausser `CURRENT_TIMESTAMP`), berechnete Spalten zwischen Treibern, `tinyblob` / `mediumblob` / `longblob` (Laravel kennt nur `blob`), Kommentare und Kollationen werden nicht verglichen.

- **Volltext** wird nur zwischen MySQL und MariaDB übertragen; nach PostgreSQL und SQLite als gewöhnlicher Index (Hinweis), von PostgreSQL ein Index auf `text` nach MySQL scheitert ohne Präfixlänge.
- **SQLite-Umbau** im Skript braucht beim Ausführen `PRAGMA foreign_keys = OFF` nur, wenn die Zielverbindung Fremdschlüssel prüft — Laravel schreibt die Pragmas dann selbst ins Skript.
- Ein Tunnel prüft die Bereitschaft, indem er einmal zum lokalen Ende verbindet — die Datenbank sieht dabei eine leere Verbindung.
- SSH-Agent, Passwort-Anmeldung bei SSH und Sprungserver (`ProxyJump`) sind bewusst nicht im Umfang.

Gelernt

- PHP merkt sich aufgelöste Pfade: `symlink()` meldet nach `rm -rf` im selben Prozess «File exists», bis `clearstatcache(true)` (Lektion im Vault).
- OpenSSH schreibt «channel N: open failed» erst ab `LogLevel=INFO` — mit `ERROR` bleibt ein unerreichbares Ziel hinter dem Tunnel stumm (Lektion im Vault).
- `ssh-keygen -p` liest die alte Passphrase von der Standardeingabe, wenn kein Terminal da ist — Geheimnisse an Hilfsprozesse über stdin, nie über die Befehlszeile (Lektion im Vault).
- Der Sprachtest entfernt Kommentare per Muster und sieht `'/**'` in einer PHP-Zeichenkette als Kommentar — Übersetzungen dazwischen galten als «nicht verwendet» (Lektion im Vault).
- Ein `sshd` als Benutzer auf einem hohen Port reicht auf macOS für echte Tunnel-Tests, ohne Remote Login einzuschalten (Lektion im Vault).